CustomerIQ AI: A Customer Intelligence Platform for E-commerce Stores

---

Submitted by

[CONFIRM: full name] (BSCS-[CONFIRM: roll no.]-[CONFIRM: section])

[CONFIRM: second student name, or delete this line] (BSCS-[roll no.]-[section])

Session 2023-2026

Supervised by

[CONFIRM: supervisor name]



Department of Computer Science

Lahore Garrison University

Lahore

CustomerIQ AI: A Customer Intelligence Platform for E-commerce Stores

A project submitted to the

Department of Computer Science

In

Partial Fulfillment of the Requirements for the

Bachelor of Science Degree in Computer Science

By

[CONFIRM: student names]

Internal Supervisor

[CONFIRM: supervisor name] _____
Designation
Department of Computer Science

External Examiner _____

Chairperson

Dr. Erum Mehmood _____
Assistant Professor
Department of Computer Science

COPYRIGHTS

This is to certify that the project titled "**CustomerIQ AI: A Customer Intelligence Platform for E-commerce Stores**" is the genuine work carried out by **[CONFIRM: student names]**, student of BSCS of Computer Science Department, Lahore Garrison University, Lahore during the academic year 2023-2026 **[CONFIRM: academic year]**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Science and that the project has not formed the basis for the award previously of any other degree, diploma, fellowship or any other similar title.

Student Name _____

Student Name _____

DECLARATION

This is to declare that the project entitled "**CustomerIQ AI: A Customer Intelligence Platform for E-commerce Stores**" is an original work done by undersigned, in partial fulfilment of the requirements for the degree "Bachelor of Science in Computer Science" at Computer Science Department, Lahore Garrison University, Lahore.

All the analysis, design and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.

Group Members

Student Name _____

Student Name _____

Supervisor

Supervisor Name _____

Date: _____

DEDICATION

[To be written by the student]

ACKNOWLEDGEMENTS

[DRAFT: please rewrite in your own words and add anyone you wish to thank]

I would like to thank my supervisor, **[CONFIRM: supervisor name]**, for guidance and feedback throughout this project, and the Department of Computer Science at Lahore Garrison University for the opportunity to carry it out.

The project was tested on the Online Retail II dataset, which Daqing Chen donated to the UCI Machine Learning Repository under a Creative Commons Attribution 4.0 licence. Without a real transaction history of that size, the segmentation, churn and cohort features could not have been checked against realistic data. The application is also built on open-source software, including Next.js, React, FastAPI, SQLAlchemy, PostgreSQL and scikit-learn, and I am grateful to the communities that maintain them.

Finally, I thank my family and friends for their patience and encouragement.

Table of Contents

Chapter 1 1
INTRODUCTION 1
1.1 Background 1
1.2 Motivation 1
1.3 Aim and Objectives 1
1.4 Scope 2
1.5 Target Users 3
1.6 Significance of the Work 3
1.7 Overview of the Solution 3
1.8 Organisation of the Report 3
Chapter 2 4
LITERATURE REVIEW 4
2.1 Customer Segmentation with RFM 4
2.2 Customer Lifetime Value 4
2.3 Churn Prediction 5
2.4 Evaluating Predictive Models 5
2.5 Existing Tools 5
2.6 Comparison with CustomerIQ AI 6
2.7 Summary 6
Chapter 3 7
PROBLEM DEFINITION 7
3.1 Problem Statement 7
3.2 Limitations of Current Practice 7
3.3 Needs of the Intended Users 7
3.4 Proposed Solution 8
3.5 Constraints Considered 8
3.6 Success Criteria 8
Chapter 4 10
SOFTWARE REQUIREMENT SPECIFICATION 10
4.1 Introduction 10
4.1.1 Purpose 10

4.1.2 Scope 10
4.1.3 Definitions 10
4.2 Overall Description 10
4.2.1 Product Perspective 10
4.2.2 User Classes 10
4.2.3 Operating Environment 11
4.2.4 Assumptions and Dependencies 11
4.3 Functional Requirements 11
4.4 Non-Functional Requirements 12
4.5 External Interface Requirements 13
4.5.1 User Interfaces 13
4.5.2 Import File Format 13
4.5.3 API Interface 13
4.5.4 Software Interfaces 14
4.6 Hardware and Software Requirements 14
4.7 Use Cases 15
Chapter 5 17
METHODOLOGY 17
5.1 Development Approach 17
5.2 Tools and Technologies 18
5.3 Design Decisions and Rationale 18
5.4 Methods and Algorithms 19
5.4.1 Import and Validation 19
5.4.2 RFM Segmentation 20
5.4.3 Health Score 20
5.4.4 Inactivity Level 20
5.4.5 Key Performance Indicators and Periods 20
5.4.6 Growth Score 21
5.4.7 Cohort Retention 21
5.4.8 Prediction Models 21
5.4.9 Recommendations 22
5.4.10 Assistant Grounding 22
5.5 Security Methodology 22
Chapter 6 24

DETAILED DESIGN AND ARCHITECTURE 24
6.1 System Architecture 24
6.1.1 Architecture Design Approach 24
6.1.2 Architecture Design 24
6.1.3 Subsystem Architecture 25
6.2 Detailed System Design 28
6.2.1 Classification 28
6.2.2 Definition 28
6.2.3 Responsibilities 28
6.2.4 Constraints 29
6.2.5 Composition 29
6.2.6 Uses and Interactions 29
6.2.7 Resources 30
6.2.8 Processing 30
6.2.9 Interface and Exports 30
6.2.10 Detailed Subsystem Design 31
Chapter 7 41
IMPLEMENTATION AND TESTING 41
7.1 Implementation Overview 41
7.2 Implementation of Core Functionality 41
7.2.1 Importing Data 41
7.2.2 Analytics and Dashboards 42
7.2.3 Prediction Models and the Quality Gate 42
7.2.4 Authentication, Sessions and Email Confirmation 42
7.2.5 AI Business Assistant 43
7.2.6 Website 43
7.2.7 Deployment 43
7.3 Testing 44
7.3.1 Testing Strategy 44
7.3.2 Automated Test Results 44
7.3.3 Selected Test Cases 45
7.3.4 Browser Test Cases 47
7.3.5 Static Checks and Build 48
7.3.6 Limits of the Testing 48

Chapter 8 49
RESULTS AND DISCUSSION 49
8.1 Import and Performance Results 49
8.2 Descriptive Analytics Results 50
8.3 Prediction Results 52
8.4 User Interface Walkthrough 53
8.5 Security and Access Results 68
8.6 AI Assistant 69
8.7 Deployment Results and Problems Met 69
8.8 Discussion 70
Chapter 9 72
CONCLUSION AND FUTURE WORK 72
9.1 Summary 72
9.2 How the Problem Was Addressed 72
9.3 Current Limitations 72
9.4 Recommendations and Future Work 73
REFERENCES 74

List of Tables

Table 2.1: How the reviewed tools and CustomerIQ AI differ, based on the public documentation cited above 6
Table 3.1: Problems identified and the design response in CustomerIQ AI 8
Table 3.2: Objectives and how they are verified 9
Table 4.1: User classes 10
Table 4.2: Functional requirements and their status 11
Table 4.3: Non-functional requirements 12
Table 4.4: API endpoints (all except health and the sign-in endpoints require a signed-in user) 13
Table 4.5: Environment used for development and hosting 14
Table 4.6: UC-01 Create an account 15
Table 4.7: UC-02 Import orders 15
Table 4.8: UC-03 Review churn risk and export a list 15
Table 4.9: UC-04 Ask the assistant 16
Table 5.1: Development stages, taken from the Git history 17
Table 5.2: Technologies used and the reason for each choice 18
Table 5.3: Segment rules (first matching rule wins) 20
Table 5.4: Quality gates for each prediction 22
Table 5.5: Threats considered and the controls used 22
Table 6.1: Subsystems and their responsibilities 25
Table 6.2: Classification of the components 28
Table 6.3: Definition of the components 28
Table 6.4: Responsibilities of the components 28
Table 6.5: Constraints of the components 29
Table 6.6: Composition of the components 29
Table 6.7: Uses and interactions of the components 29
Table 6.8: Resources used by the components 30
Table 6.9: Processing performed by the components 30
Table 6.10: Interfaces of the components 30
Table 7.1: Size of the code base (counted from the files, excluding libraries) 41
Table 7.2: Back-end test suite results by file 44
Table 7.3: Selected automated test cases 45

Table 7.4: Browser test cases and observed results 47
Table 8.1: Result of importing the public dataset 49
Table 8.2: Measured response times on the public dataset (single run, development computer). 49
Table 8.3: Overview figures for the 90 days ending 9 December 2011 50
Table 8.4: Customer segments on the public dataset (revenue to date) 51
Table 8.5: Distribution of health scores 51
Table 8.6: Prediction models on the public dataset 52
Table 8.7: Assessment of the objectives 70

List of Figures

Figure 6.1: Deployment architecture of CustomerIQ AI 25
Figure 6.2: Component diagram of the website and the API 26
Figure 6.3: Data flow diagram, level 0 (context) 27
Figure 6.4: Data flow diagram, level 1 27
Figure 6.5: Use case diagram 32
Figure 6.6: Entity relationship diagram 33
Figure 6.7: Database diagram 34
Figure 6.8: Class diagram 35
Figure 6.9: Activity diagram: importing orders 36
Figure 6.10: Activity diagram: obtaining predictions 37
Figure 6.11: Sequence diagram: account creation with email confirmation 38
Figure 6.12: Sequence diagram: loading the overview 39
Figure 6.13: State machine diagram: account 39
Figure 6.14: State machine diagram: import job 40
Figure 6.15: State machine diagram: prediction model 40
Figure 8.1: Landing page shown to visitors who are not signed in 53
Figure 8.2: Sign-up form (shown when email confirmation is configured) 53
Figure 8.3: Code screen after a wrong code was entered 54
Figure 8.4: Overview dashboard with the key figures, revenue trend and segment mix 55
Figure 8.5: Period calendar with quick choices and the comparison period 56
Figure 8.6: Customer segmentation page 57
Figure 8.7: Churn prediction page with the model status, risk distribution and customer list 58
Figure 8.8: Lifetime value page: the prediction is withheld and the reason is shown 59
Figure 8.9: Health score page with the formula, distribution and drivers 60
Figure 8.10: Revenue intelligence page: the forecast is withheld and the revenue drivers are shown with order statistics 61
Figure 8.11: Cohort retention table 62
Figure 8.12: Recommendations page 63
Figure 8.13: Customer page with figures, health factors, recommended action and order history 64
Figure 8.14: Import page after a file with two bad rows: nothing is saved and the problems are listed without repeating the values 65

Figure 8.15: Currency menu with data in pounds shown in rupees, with the rate and its date..... 66
Figure 8.16: Interface in Urdu; long explanatory texts remain in English 66
Figure 8.17: AI business assistant page, showing the provider status and what the assistant can see 67
Figure 8.18: Overview on a 390-pixel-wide phone screen 68

List of Abbreviation

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the ROC Curve
CLV	Customer Lifetime Value
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity Relationship
FYP	Final Year Project
GBP	Pound Sterling
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MAE	Mean Absolute Error
ML	Machine Learning
ORM	Object-Relational Mapper
RFM	Recency, Frequency, Monetary value
RLS	Row-Level Security
ROC	Receiver Operating Characteristic
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SRS	Software Requirements Specification
TLS	Transport Layer Security
UCI	University of California, Irvine (Machine Learning Repository)
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
XLSX	Microsoft Excel workbook format

ABSTRACT

Small online stores usually own a detailed record of what their customers have bought, yet they rarely have a practical way to turn that record into decisions: which customers are drifting away, which ones are worth a personal offer, and how much of next quarter's revenue depends on a handful of buyers. Commercial platforms answer these questions, but they are typically tied to a particular store platform or plan, and the predictions they show are not accompanied by evidence of how accurate they are.

This project presents CustomerIQ AI, a web application that analyses an order history imported from an Excel workbook, a comma-separated text file or a Shopify export. The system groups customers into five segments using recency, frequency and spend; calculates a customer health score, revenue trends, cohort retention and a transparent growth score; and shows churn probability, predicted lifetime value and a revenue forecast only when a model has passed a test. Each model is trained on an older period and tested on a later period it has not seen, and it must outperform a simple rule of thumb. Otherwise the page states that the figure is not available yet and gives the reason. An optional assistant answers questions using only a summary of the computed results. Access requires an account, and new accounts confirm their email address with a six-digit code.

The application was built with a Next.js and TypeScript front end, a FastAPI and Python back end, and PostgreSQL, and it runs both on a local computer and on the free tiers of Vercel and Supabase. It was evaluated on the public Online Retail II dataset (36,594 orders from 5,852 customers). The churn model scored 0.80 on unseen data (area under the ROC curve) against 0.76 for the recency baseline, while the lifetime value and revenue forecast models were correctly withheld because they did not beat their baselines. The automated suite of 245 back-end tests passes, and the main user flows were exercised in a real browser.

Current limitations include a single shared data set for all users, no direct connection to store platforms, and unresolved email delivery from the hosted deployment. Future work includes per-account data separation, platform integrations and a wider range of validated models.

Chapter 1

INTRODUCTION

1.1 Background

Every online store collects the same raw material: a list of orders, each with a customer, a date and an amount. Hidden in that list are answers to questions that matter to the owner. Who are my best customers? Who bought regularly last year and has gone quiet? Which month depends on one or two large orders? In a small business these questions are usually answered by instinct, or by exporting the data to a spreadsheet and sorting it by hand.

Customer analytics software exists to answer such questions, and the ideas behind it are well established. Customers can be grouped by how recently, how often and how much they buy; their expected future value can be estimated; and the likelihood that they stop buying can be predicted. The difficulty for a small store is not the lack of methods but the lack of a tool that is simple to start with, does not require a particular store platform, and is honest about how far its numbers can be trusted.

1.2 Motivation

CustomerIQ AI was motivated by two observations. The first is that many dashboards present every figure with the same confidence, whether it comes from a simple sum or from a prediction trained on a few hundred rows. A store owner has no way to tell the difference. The second is that analytics tools often assume an integration with a specific platform or a long order history, which makes them hard to try on a small data set or an exported file.

The project therefore set out to build a complete but modest system around one rule: nothing should be shown as a fact unless it was calculated from the imported data, and nothing should be shown as a prediction unless a model has been tested on data it did not see. When that is not possible, the interface says so and explains why.

1.3 Aim and Objectives

The aim of the project is to build a web application that turns an imported order history into clear customer insights for the owner of an online store, while stating plainly when the data is not sufficient for a reliable prediction.

To achieve this aim, the following objectives were set:

1. Import order data from CSV and Excel files, and from the Shopify orders export, with validation that reports every rejected row and the reason.
2. Calculate descriptive analytics directly from the data: key performance indicators for a chosen period, five customer segments, a customer health score, revenue drivers, cohort retention and rule-based recommendations.
3. Provide predictions (churn probability, predicted lifetime value and a revenue forecast) using models that are validated on a later, unseen period and compared with a simple baseline, and withhold any prediction that fails.
4. Offer an assistant that answers questions in plain language using only the computed results, without inventing figures.
5. Protect the data with accounts, secure sessions and email confirmation of new accounts.
6. Make the application easy to use: a landing page, search and filters, CSV export, a calendar for choosing the period, a currency menu and a choice of interface languages.
7. Keep the system deployable on free hosting so that it can be demonstrated without a payment card.

1.4 Scope

In scope

- A web application with a Next.js front end and a FastAPI back end, using a PostgreSQL database.
- Imports from files only. The Shopify orders export is converted automatically; other platforms can be used through the standard column layout.
- Analytics and predictions computed on the server from the stored orders, on demand.
- Account creation with email confirmation, sign-in, sign-out and password change.
- Local use on one computer and hosted use on Vercel with Supabase.

Out of scope

- Live synchronisation with Shopify, WooCommerce or customer relationship management (CRM) systems. The interface lists these as “not built yet”.
- Sending campaigns or messages to customers.
- Separate data for each account. In the current version all signed-in users see the same imported data.
- Right-to-left page layout for Urdu and Arabic, and translation of every long explanatory paragraph.

1.5 Target Users

The intended users are owners and analysts of small and medium online stores who can export their orders from the store platform but do not have a data team. A second audience is a student or evaluator who wants to see how segmentation, churn and cohort analysis behave on a real public data set. The application does not assume any statistical background: every score is accompanied by its definition, and every model by its measured accuracy.

1.6 Significance of the Work

The contribution of the project is a combination rather than a single new technique. The individual methods (RFM segmentation, gradient boosting, cohort tables) are standard. What the project adds is the discipline of applying them under explicit honesty rules: time-based validation, quality gates against baselines, missing values kept missing, and a visible “not available yet” state. This was checked on a real data set, where two of the three prediction models were correctly withheld, which shows that the gates work in practice and not only in theory.

1.7 Overview of the Solution

The solution has three parts. The website, built with Next.js, presents the dashboards and the import page. The API, built with FastAPI, validates and stores orders, calculates analytics, trains and tests the models, and handles sign-in. A PostgreSQL database stores the customers, orders, import records, accounts and sessions. The website can also run without the API in a labelled demonstration mode that shows sample data and is never mixed with real data.

1.8 Organisation of the Report

Chapter 2 reviews the literature and existing tools. Chapter 3 defines the problem. Chapter 4 gives the software requirements. Chapter 5 describes the methodology and the main design decisions. Chapter 6 presents the architecture and detailed design with diagrams. Chapter 7 explains the implementation and testing. Chapter 8 reports the results on the public data set and discusses the limitations. Chapter 9 concludes the report and suggests future work. The references follow.

Chapter 2

LITERATURE REVIEW

This chapter reviews the concepts on which the application is built and the existing tools that address the same need. It ends by stating how the implemented system relates to them.

2.1 Customer Segmentation with RFM

Segmentation divides customers into groups that can be treated differently. One of the oldest and most common approaches in retail is RFM analysis, which scores each customer on recency (how recently they bought), frequency (how often) and monetary value (how much). Chen, Sain and Guo [2] applied an RFM-based segmentation to the transactions of a UK online retailer and combined it with clustering to describe customer groups. Their data, later published as the Online Retail data sets, is the same family of data used to test this project [9].

RFM has two practical advantages for a small store: it needs only three facts that every order file contains, and its results can be explained in one sentence. CustomerIQ AI uses a rule-based version of RFM that is described in Chapter 5, rather than a clustering algorithm, so that the segment of any customer can be explained and reproduced.

2.2 Customer Lifetime Value

Customer lifetime value (CLV) estimates the total contribution of a customer to the business over the relationship. Gupta et al. [3] review the main approaches to modelling it. In non-contractual settings such as online retail, where the retailer cannot observe when a customer has “left”, probabilistic models of repeat purchasing are common. Fader, Hardie and Lee [1] proposed a widely used model of this kind, which estimates purchase rate and the probability of being “alive” from just a few summary measures of each customer’s history.

CustomerIQ AI takes a different and simpler route. It reports the revenue each customer has already generated, and it adds a machine-learning estimate of spending over the next 180 days only when that estimate is more accurate than a plain baseline (assuming that customers repeat their last 180 days). This is a deliberate trade-off: the probabilistic models are more theoretically grounded, but the chosen approach shares its features with the churn model and its accuracy can be tested in exactly the same way.

2.3 Churn Prediction

Churn prediction estimates which customers are about to stop buying so that a retention action can be targeted. Much of the published work concerns subscription businesses such as telecommunications. Verbeke et al. [4] compare many classification techniques for telecom churn and argue that models should be judged on the value of the decisions they support, not only on statistical accuracy. In an online store there is no cancellation event, so churn must be defined from behaviour. In this project a customer is counted as churned if they place no order in the next 90 days. This is a modelling choice, and it is shown on the Churn Prediction page.

2.4 Evaluating Predictive Models

A prediction is only useful if its quality is known. Three ideas from the literature shape the evaluation in this project.

- **Ranking quality.** The receiver operating characteristic (ROC) curve and the area under it (AUC) measure how well a model ranks customers who churn above customers who do not [7]. A value of 0.5 is no better than chance.
- **Probability quality.** The Brier score [8] is the mean squared difference between predicted probabilities and actual outcomes. Lower is better, and it penalises confident wrong predictions.
- **Honest validation.** For data that unfolds over time, a model should be trained on an earlier period and tested on a later one, so that it never sees the future. A random split would let information from the future leak into training.

The model family used for churn and spending is gradient boosting of decision trees [5]. The implementation is the histogram-based gradient boosting estimators of scikit-learn [6, 18], which handle missing values natively. This property is useful here because optional fields such as engagement scores are often absent.

2.5 Existing Tools

Several commercial platforms offer customer analytics to online stores. Two were examined through their public documentation, to understand what such tools provide and how they describe it.

Klaviyo

Klaviyo's help centre describes predictive analytics that include historic, predicted and total CLV, a churn risk prediction based on order count and frequency, the average time between orders, and the expected date of the next order [16]. The same page lists conditions for the feature to appear: at least 500 customers who have placed an order, an e-commerce integration or order events sent through its API, at least 180 days of order history with orders in the last 30 days, and some customers with three or more orders. The page states that predictions work best when averaged

over many customers and are not expected to be exact for any single person. It does not publish accuracy figures for the models.

Shopify

Shopify's admin lets a merchant group customers into segments using filters, operators and values, and it provides templates and a query language as a starting point [17]. This is a flexible way to define a segment that the merchant already has in mind. The cited page describes segments as rules chosen by the merchant and does not describe predictions, so questions such as model accuracy do not arise there.

2.6 Comparison with CustomerIQ AI

Table 2.1: How the reviewed tools and CustomerIQ AI differ, based on the public documentation cited above

Aspect	Klaviyo [16]	Shopify segments [17]	CustomerIQ AI (this project)
Data source	Store integration or API events	The merchant's own Shopify store	Imported CSV, Excel or Shopify export
Predictive features	Predicted CLV, churn risk, next order date	Segments defined by the merchant with filters	Churn probability, predicted CLV, 3-month forecast
Minimum data	Stated (for example 500 purchasers, 180 days)	Not applicable	Checked by the models, with the shortfall shown in words
Accuracy disclosure	Not published on the cited page	Not applicable	AUC, Brier score and baseline shown next to each model
Withholds weak predictions	Hides the section until requirements are met	Not applicable	Hides any model that fails its test, with the reason
Platform dependency	Needs a supported integration	Shopify only	Platform neutral
Maturity	Commercial product	Commercial product	Student project, tested on one public data set

The comparison is limited to what the cited pages say. It is not a claim that the commercial tools are less accurate: they are mature products used at a scale this project has not been tested at. The point of the table is to show which design choices CustomerIQ AI makes differently: it works from a file, and it reports how good each model is.

2.7 Summary

The methods behind the project are standard, and the commercial tools show that the need is real. The gap that CustomerIQ AI addresses is narrower: a platform-neutral tool that works from an export and publishes the evidence for each prediction. The following chapters describe the problem more precisely and the way it was solved.

Chapter 3

PROBLEM DEFINITION

3.1 Problem Statement

The owner of a small online store holds a record of every order but cannot easily tell which customers are valuable, which are leaving, and how reliable any forecast is. The tools that could answer these questions are either tied to a store platform or a minimum amount of history, or they present predictions without evidence of their accuracy. As a result, decisions about whom to contact, and with what offer, are made without reliable information.

3.2 Limitations of Current Practice

- **Manual analysis.** Sorting an exported order file by hand does not scale beyond a few hundred customers, and it cannot rank customers by their likelihood of leaving.
- **Platform dependence.** Predictive features in commercial tools are often available only through a supported integration. For example, the Klaviyo page cited in Chapter 2 requires an e-commerce integration or API events [16]. A store that only has a file export cannot use them.
- **Minimum data conditions.** The same page requires at least 500 customers who have ordered and 180 days of history [16]. Smaller stores fall below these limits and see nothing.
- **Predictions without evidence.** The cited documentation says predictions are imprecise for individuals but does not publish accuracy figures [16]. A user cannot judge how much weight to give a "62% chance of leaving".
- **Silent failure of weak models.** A model trained on too little data can still return a number for every customer. Unless the tool checks the model, the number looks as authoritative as a sum.

3.3 Needs of the Intended Users

- Start from a file, with no integration, and see useful results within minutes.
- Understand the meaning of every number without statistical training.
- Know when a figure is a measurement and when it is an estimate, and how good the estimate has been on past data.
- Find a specific customer quickly, see their orders, and export a list to hand to a marketing person.

- Read the interface in a familiar language and see amounts in a familiar currency.
- Keep the data behind a login, since customer lists are sensitive.

3.4 Proposed Solution

CustomerIQ AI addresses each limitation with a specific design response, summarised in Table 3.1.

Table 3.1: Problems identified and the design response in CustomerIQ AI

Problem	Design response
Manual analysis does not scale	Server-side calculation of segments, health scores, KPIs and cohorts from the imported orders; filters, search and CSV export on the results.
Platform dependence	Import from CSV, Excel or a Shopify export. Standard column names are used so that any platform that can export orders can be used.
Minimum data conditions	The models check the data themselves. If it is insufficient, the page says what is missing, for example the number of days of history needed.
Predictions without evidence	Every prediction is shown with the AUC, Brier score and baseline score measured on a later, unseen period.
Silent failure of weak models	A quality gate withholds a model that is not better than a simple baseline. The page shows “Not available yet” and the reason.
Sensitive data	Accounts, secure session cookies, email confirmation of new accounts, and row-level security on all tables.
Language and currency	Twelve interface languages for short texts, and a currency menu that converts amounts with published exchange rates and states the rate date.

3.5 Constraints Considered

- The project had to run without a payment card, so the free tiers of Vercel and Supabase were used. These tiers have limits that affect the design. For example, a Vercel Function accepts a request body of at most 4.5 MB [15].
- No real customer data could be used for testing. The public Online Retail II dataset was converted to the import format [9], and it contains no personal details beyond customer numbers.
- The author could not test integrations with a real store, which is why platform synchronisation was left out of scope rather than built without verification.

3.6 Success Criteria

The project is judged against its objectives. Table 3.2 states how each objective is verified, and where the evidence is reported.

Table 3.2: Objectives and how they are verified

Objective	Verification	Reported in
O1 Import with validation	Parsing and API tests; import of the full public dataset; bad-row import in the browser	Sections 7.3, 8.1
O2 Descriptive analytics	Formula tests; the dashboards on the public dataset	Sections 7.3, 8.2
O3 Validated predictions	Prediction tests; measured AUC, Brier score and baselines	Sections 7.3, 8.3
O4 Assistant	Tests with simulated providers; privacy test of the context	Sections 7.3, 8.6
O5 Access control	Authentication, security and verification tests; browser flow	Sections 7.3, 8.5
O6 Usability	Browser walk-through of calendar, currency, language, search and export	Section 8.4
O7 Free hosting	Deployment on Vercel and Supabase; hosted status checks	Section 8.7

Chapter 4

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Introduction

4.1.1 Purpose

This chapter specifies what CustomerIQ AI does and the conditions under which it works. Each requirement is marked with its actual status at the time of writing, so that the specification matches the implemented system and not an intended one.

4.1.2 Scope

The specification covers the website, the API and the database. It does not cover the hosting platforms, which are treated as external services.

4.1.3 Definitions

Order: one purchase, identified by an order ID, with a customer, a date and a value. **Segment**: one of five groups produced by RFM scoring. **As-of date**: the date of the latest order in the data, used as "today" for all calculations. **Quality gate**: a test that a model must pass before its predictions are shown. **Demo mode**: running the website without an API, with labelled sample data.

4.2 Overall Description

4.2.1 Product Perspective

CustomerIQ AI is a self-contained web application. The website talks only to its own API. The API reads and writes the database and, when configured, calls an email service and an AI provider. The browser separately calls a public exchange-rate service to convert displayed amounts.

4.2.2 User Classes

Table 4.1: User classes

User class	Description and permissions
Visitor	Not signed in. Can see the landing page and the sign-in and sign-up pages. All data pages and API calls are refused.
Signed-in user	Has a confirmed account. Can import data, use every dashboard, download lists, use the assistant, change their password. All signed-in users see the same data.

User class	Description and permissions
Site owner	The person who creates the first account and runs the site. Uses the same interface as other users, plus command-line scripts to create, reset or deactivate users and to migrate the database.

4.2.3 Operating Environment

The website runs in any current browser. The API needs Python 3.13 and PostgreSQL. The tools used during development are listed in Section 4.6.

4.2.4 Assumptions and Dependencies

- The imported file has one row per order and the required columns (Section 4.5).
- Order dates are ISO dates (YYYY-MM-DD) or real spreadsheet dates, and order values are plain numbers.
- All values in one file are in one currency. The application does not know which one, so the owner states it in the currency menu.
- Hosted use depends on Vercel, Supabase, an email route and, optionally, an AI provider.

4.3 Functional Requirements

Table 4.2 lists the functional requirements. Status values are Implemented (built and covered by tests or by the browser walk-through), Partial (built with a stated limitation) and Not implemented.

Table 4.2: Functional requirements and their status

ID	Requirement	Status
FR-01	A visitor who is not signed in and opens the site address sees a landing page.	Implemented
FR-02	The first account (owner) can be created, optionally protected by a setup code.	Implemented
FR-03	Anyone with a real email address can sign up when email is configured. The address is confirmed with a six-digit code that expires after 15 minutes and allows five wrong tries.	Implemented; delivery from the hosted site unresolved (see 8.7)
FR-04	A user can sign in with email and password. “Keep me signed in” keeps the session for 30 days, otherwise 24 hours.	Implemented
FR-05	Sign-in for an email is locked for 15 minutes after 8 wrong passwords.	Implemented (counter kept in server memory)
FR-06	A user can change their password; other browsers are signed out.	Implemented
FR-07	A user can sign out, which ends the session.	Implemented
FR-08	The owner can list, create, reset and deactivate users with a script.	Implemented
FR-09	A user can upload a CSV or XLSX file of up to 10 MB and 50,000 data rows (4.5 MB on the hosted site).	Implemented
FR-10	The browser previews a CSV file and checks its columns before upload.	Implemented (CSV only)
FR-11	The server validates every row, saves valid rows, and lists rejected rows with the reason, without repeating the data.	Implemented
FR-12	Uploading an order ID that already exists updates the order instead of duplicating it.	Implemented

ID	Requirement	Status
FR-13	A Shopify orders export is detected and converted to orders.	Implemented
FR-14	Import records can be listed and read through the API.	Implemented (API only)
FR-15	The overview shows total customers, revenue, average CLV, retention rate, churn risk and a growth score, each compared with the previous period.	Implemented
FR-16	The user can choose any reporting period up to three years with a calendar.	Implemented
FR-17	Customers are placed in five RFM segments when there are at least 20 customers.	Implemented
FR-18	Each customer has a 0–100 health score from four factors; a missing engagement factor is left out and the weights are rescaled.	Implemented
FR-19	Revenue pages show monthly revenue, revenue by category, channel and country (with order statistics), new and returning customers, and a revenue-at-risk calculator.	Implemented
FR-20	A cohort table shows retention by first-order month.	Implemented
FR-21	Rule-based recommendations are generated per segment, without invented revenue estimates.	Implemented
FR-22	A customer page shows figures, health factors, the recommended action and the order history.	Implemented
FR-23	Customer lists can be searched, filtered and downloaded as CSV.	Partial (lists show the first 500 customers)
FR-24	Churn probability for the next 90 days is shown only if the model passes its gates.	Implemented
FR-25	Predicted CLV over the next 180 days is shown only if the model passes its gates.	Implemented
FR-26	A three-month revenue forecast with an 80% range is shown only if it passes its gates.	Implemented
FR-27	Models are retrained automatically when the data changes.	Implemented
FR-28	Accuracy figures, or the reason a model is unavailable, are shown with each prediction.	Implemented
FR-29	An assistant answers questions using only a computed data summary; the provider key stays on the server.	Implemented; tested with simulated providers
FR-30	The assistant receives no customer names or email addresses.	Implemented
FR-31	Amounts can be shown in nine currencies using published exchange rates.	Implemented
FR-32	The interface can be shown in twelve languages.	Partial (short texts only; no right-to-left layout)
FR-33	Without an API address the site runs in a labelled demo mode with sample data.	Implemented
FR-34	The layout adapts to phone width.	Implemented
FR-35	Live synchronisation with Shopify, WooCommerce and CRM systems.	Not implemented
FR-36	Separate data for each account.	Not implemented

4.4 Non-Functional Requirements

Table 4.3: Non-functional requirements

Category	Requirement and how it is met
Security: passwords	Passwords are stored only as scrypt hashes (N = 2^15, r = 8, p = 3, random salt) and must be at least 10 characters and not on a list of common passwords. These parameters correspond to one of the equivalent settings listed in the OWASP guidance [10, 11].

Category	Requirement and how it is met
Security: sessions	A random token is sent in an HttpOnly, SameSite=Lax cookie, marked Secure in production. Only a SHA-256 hash of the token is stored.
Security: requests	Writes whose Origin header is not on the allow-list are refused. API documentation is hidden in production. Row-level security is enabled on every table [14].
Security: privacy	Error messages and import records never repeat imported values. The assistant context contains no names or emails. Provider keys are never sent to the browser.
Honesty	Measured figures and predictions are visibly different. A missing value is shown as missing, never as zero. A failing model reports "unavailable" instead of an error or a guess.
Usability	Plain-language labels and definitions, a skip link, labelled controls and visible keyboard focus. These measures were implemented but no formal accessibility audit was carried out.
Performance	No numerical targets were set in advance. Measured response times on the public dataset are reported in Section 8.1.
Portability	Runs on Windows with a local database, and on Vercel with Supabase.
Maintainability	Six database migrations, typed API schemas, a linted TypeScript code base and an automated test suite of 245 tests.

4.5 External Interface Requirements

4.5.1 User Interfaces

The site has the following pages: landing, sign-in and sign-up, overview, segmentation, churn prediction, cohort retention, lifetime value, health scores, revenue intelligence, recommendations, AI business assistant, import data, customer page, and account. Examples are shown in Chapter 8.

4.5.2 Import File Format

Required columns are `customer_id`, `email`, `order_id`, `order_date` and `order_value`. Optional columns are `first_name`, `last_name`, `country`, `product_category`, `channel` and `engagement` (0–100). Header names are not case-sensitive. Files up to 10 MB are accepted, and a Shopify orders export is recognised by its own header.

4.5.3 API Interface

The browser and the website call the REST API under `/api/v1`. Table 4.4 lists the endpoints.

Table 4.4: API endpoints (all except health and the sign-in endpoints require a signed-in user)

Method	Path	Purpose
GET	/health	Status of the API and the database
GET	/auth/status	Whether setup or sign-up is open, and whether the caller is signed in
POST	/auth/setup, /auth/signup	Create the owner account, or an ordinary account (unconfirmed)
POST	/auth/verify, /auth/resend	Confirm the email with the code, or request a new code
POST	/auth/login, /auth/logout	Sign in and sign out

Method	Path	Purpose
GET	/auth/me	The current user
POST	/auth/change-password	Change password
POST / GET	/imports, /imports, /imports/{id}	Upload a file; list imports; read one import
GET	/dashboard/overview	KPIs, summary, segments, revenue series (period by start and end dates)
GET	/customers, /customer	Customer list; one customer with orders
GET	/segments	Segment summaries
GET	/predictions/churn, /predictions/clv	Churn and lifetime value views
GET	/customers/health	Health score distribution and customers
GET	/revenue/intelligence	Revenue series, drivers and revenue at risk
GET	/cohorts	Cohort retention table
GET	/recommendations	Rule-based recommendations
GET / POST	/assistant/status, /assistant/chat	Assistant availability; ask a question

4.5.4 Software Interfaces

- PostgreSQL through SQLAlchemy and the psycopg driver.
- Exchange rates from a public HTTPS service, requested by the browser.
- Anthropic Claude (default) or OpenAI for the assistant, requested only by the API.
- Email through SMTP, Brevo's HTTPS API or a Google Apps Script relay, selected by configuration.

4.6 Hardware and Software Requirements

Table 4.5: Environment used for development and hosting

Item	Details
Development computer	Windows 10 Pro (build 18362). [CONFIRM: processor and memory]
Runtimes	Python 3.13, Node.js 24, PostgreSQL 18
Editor and tools	Visual Studio Code, Git and GitHub (private repository), Google Chrome, an AI coding assistant (Claude Code) [CONFIRM: how to describe AI assistance under your university's policy]
Back-end libraries	FastAPI 0.143, SQLAlchemy 2.1, Alembic 1.20, psycopg 3.3, Pydantic 2.14, openpyxl 3.1, scikit-learn 1.9, NumPy 2.5, SciPy 1.18, anthropic 1.14, openai 3.31
Front-end libraries	Next.js 16.4, React 19.3, TypeScript 5, Tailwind CSS 4, Recharts 3.10
Hosting (optional)	Vercel Hobby plan, two projects (website and API); Supabase free project (PostgreSQL). Documented Vercel limits include a 4.5 MB request body, a 300-second maximum duration on the Hobby plan, and a 500 MB limit for Python functions [15].
User device	Any recent browser; no installation.

4.7 Use Cases

The use case diagram is Figure 6.5 in Section 6.2.10. Tables 4.6 to 4.9 describe four of the use cases in detail.

Table 4.6: UC-01 Create an account

Field	Description
Actor	Visitor
Precondition	Sign-up is open (email is configured and an owner account exists).
Main flow	1. The visitor opens the sign-up form and enters name, email and a password. 2. The system checks the email format and the password rules and creates an unconfirmed account. 3. The system emails a six-digit code and shows the code screen. 4. The visitor enters the code. 5. The system marks the email as confirmed, starts a session and opens the dashboard.
Alternative flows	Wrong code: an error is shown and the attempt is counted (five allowed). Expired code or no email: the visitor requests a new code after 60 seconds. Email already confirmed: the visitor is told to sign in.
Postcondition	The account is confirmed and signed in, or remains unconfirmed and cannot see data.

Table 4.7: UC-02 Import orders

Field	Description
Actor	Signed-in user
Precondition	The user holds an order file in CSV, XLSX or Shopify export format.
Main flow	1. The user chooses a file on the Import page. 2. The browser shows the number of rows, the columns and a preview. 3. The user presses Import customers. 4. The server validates every row and saves the valid ones. 5. The page shows the numbers of new, updated and rejected rows and the reasons for rejections.
Alternative flows	Unsupported type, empty file or file over the limit: refused before upload with the reason. Required column missing: nothing is saved.
Postcondition	Valid orders are stored; dashboards use them at the next load; models retrain when needed.

Table 4.8: UC-03 Review churn risk and export a list

Field	Description
Actor	Signed-in user
Precondition	Orders have been imported.
Main flow	1. The user opens Churn Prediction. 2. The page shows the model status and accuracy, or the reason it is unavailable. 3. The user searches or filters the customer list. 4. The user presses Download CSV and receives the filtered list.
Alternative flows	If the churn model is unavailable, the page shows rule-based inactivity levels and says they are not predictions.
Postcondition	A CSV file with the shown customers is saved on the user’s computer.

Table 4.9: UC-04 Ask the assistant

Field	Description
Actor	Signed-in user
Precondition	Data has been imported and an AI provider key is configured on the server.
Main flow	1. The user types a question. 2. The API builds a summary of the computed results without names or emails. 3. The API sends the summary and the recent conversation to the provider. 4. The answer is shown with the data date and clear wording for estimates.
Alternative flows	No key: the page says the assistant is not configured. Provider error or limit: a specific message is shown. No data: a fixed reply is given without calling the provider.
Postcondition	No data is stored by the application beyond the conversation shown on screen.

Chapter 5

METHODOLOGY

This chapter describes how the system was developed, which tools and design decisions shaped it, and the methods used for each calculation. The calculations are described exactly as they are implemented, so that every number on a dashboard can be traced back to a rule.

5.1 Development Approach

The project was developed incrementally. The work was divided into stages, each of which delivered a working piece of the system, was covered by tests, and was committed to the Git repository before the next stage began. No formal framework such as Scrum was followed, and no sprint planning or backlog tool was used. The Git history therefore serves as the record of the process. Between 14 and 16 October 2026 it contains 43 commits, summarised in Table 5.1.

Table 5.1: Development stages, taken from the Git history

No.	Stage and outcome	Date (2026)	Main layers
1	Baseline Next.js front end with clearly labelled sample data; accessibility fixes for the mobile navigation and KPI change indicators	14 Oct	Front end
2	FastAPI skeleton, PostgreSQL connection, migrations and health check	14 Oct	Back end, database
3	Server-side import of CSV and XLSX files with row validation and an upload page	15 Oct	Back end, front end
4	Descriptive analytics API; dashboards connected to live data	15 Oct	Back end, front end
5	AI business assistant grounded in the computed data (Claude by default, OpenAI optional); Shopify export support	15 Oct	Back end
6	Prediction models with time-based validation and quality gates; conversion script for the public dataset; fix for data without engagement values	15 Oct	Back end
7	Accounts, sessions, sign-in page and protected API	15 Oct	Back end, front end
8	Deployment guide, row-level security, database SSL options, growth score, one-click launcher	15 Oct	All
9	Currency menu with live rates; Urdu and English toggle, later twelve languages	15 Oct	Front end
10	CSV download, channel and country statistics, cohort retention, period calendar, customer page	16 Oct	All
11	Landing page; email confirmation of new accounts with several delivery options	16 Oct	All

Two working rules were agreed before the code was written and applied throughout. First, changes to the database, permissions and the repository were explained and approved before they were

made. Second, the honesty rules of Section 5.3 were treated as requirements, so that features were designed to show “not available” rather than an unreliable value.

5.2 Tools and Technologies

Table 5.2: Technologies used and the reason for each choice

Layer	Technology	Reason
Front end	Next.js 16 (App Router) [13], React 19, TypeScript, Tailwind CSS 4	Server components fetch data before the page is sent and stream slow parts, while client components handle interaction. TypeScript catches mismatches with the API types.
Charts	Recharts	Declarative charts that integrate with React and need no separate chart server.
Back end	Python 3.13, FastAPI [12], Pydantic	Typed request and response models give automatic validation. Python also holds the machine-learning libraries, so the API and the models share one language.
Database	PostgreSQL 18 locally, Supabase (PostgreSQL) when hosted; SQLAlchemy 2 and Alembic	A relational database fits customers, orders and accounts. Alembic migrations record every schema change.
Machine learning	scikit-learn (histogram gradient boosting), NumPy, SciPy	Handles missing values natively and trains in about a second on the test data, so models can be retrained whenever data changes.
Spreadsheets	openpyxl	Reads XLSX files without a spreadsheet application.
AI assistant	Anthropic Claude (default), OpenAI (optional)	The assistant is a thin layer over the computed summary. Keys are stored only on the server.
Testing	pytest, FastAPI test client, ESLint, TypeScript compiler, Playwright (browser walk-through)	Automated tests for the back end; static checks for the front end; a real browser for user flows.
Hosting	Vercel (two projects), Supabase	Both have free tiers that do not require a payment card.

5.3 Design Decisions and Rationale

The decisions below explain why the system is built the way it is. Most of them trade some convenience for honesty or simplicity.

1. **Compute on demand from the orders.** Segments, scores and KPIs are not stored. They are recalculated from the order table at each request. This keeps the results consistent after every import and avoids stale tables. The cost is that each request reloads the orders, which is acceptable at the tested size (Section 8.1).
2. **The latest order is “today”.** All time-based figures are measured from the date of the latest order (the as-of date). An old file is therefore analysed relative to its own end, and “last 90 days” means the 90 days before the data ends.
3. **Honesty rules for predictions.** Every model is trained on an older period and tested on a later one. It must reach a minimum quality and beat a simple baseline. Missing values stay

missing. If a model fails, the page says so and gives the reason. No sample numbers are ever mixed into live pages.

4. **Rule-based segmentation and recommendations.** The segment and the recommended action of every customer follow fixed, documented rules, so they can be explained to a store owner.
5. **The browser talks to its own site.** Calls to `/api/v1` are forwarded by the website to the API. The session cookie therefore belongs to the website and works on any hosting, while server-side pages call the API directly and forward the cookie.
6. **Server-side sessions rather than tokens in the browser.** A random token in an HttpOnly cookie, whose hash is stored in the database, can be revoked. This is what allows "sign out other devices" after a password change.
7. **Demo mode kept separate.** Without an API address the website shows sample data with a visible label, and sign-in is not required. With an API address it never falls back to sample data after an error.
8. **One shared data set.** To keep the first version simple, accounts control access, not ownership. This is recorded as a limitation (Section 8.8).
9. **Currency conversion in the browser.** Stored values are never changed. The owner states the currency of the file and the currency to display, and the browser multiplies by a published rate and shows its date. If the rate cannot be loaded, amounts are shown in the original currency and labelled as such.
10. **Translation by lookup.** Interface text is written in English and replaced by a translation if one exists. Untranslated text stays in English, so a missing translation never leaves a blank.
11. **Several email routes.** The hosted deployment could not reach SMTP servers (Section 8.7), so the mailer supports SMTP, Brevo's HTTPS API and a Google Apps Script relay, chosen by configuration.

5.4 Methods and Algorithms

5.4.1 Import and Validation

A file is accepted if it is a CSV or XLSX file of at most 10 MB. The server detects whether it is a standard file or a Shopify orders export. For a Shopify export, line items are grouped into orders, the customer's email is used as the customer ID, and cancelled, voided and fully refunded orders are skipped while partial refunds are subtracted. Every row is then validated: required values must be present, the email must look like an email, the date must be a real date that is not in the future, and the value must be a non-negative number that fits the database. A row that fails is rejected with the row number, the column and a message that describes the problem without repeating the value. An order ID that already exists is updated instead of duplicated. The import record keeps the counts and the first 200 problems.

5.4.2 RFM Segmentation

Each customer receives a recency score, and a combined frequency and monetary score, from 1 to 5. Recency is the number of days since the last order, and fewer days give a higher score. For each of the three measures, the customer's percentile among all customers is calculated with mid-ranks, so that tied values share one percentile, and the percentile is cut into five equal bands. The frequency and monetary scores are averaged and rounded half up to give the combined score. The first matching rule in Table 5.3 assigns the segment. Segments are produced only when there are at least 20 customers, because a ranking among fewer customers is not meaningful.

Table 5.3: Segment rules (first matching rule wins)

Segment	Rule	Recommended action
Champions	Recency ≥ 4 and frequency-monetary ≥ 4	Invite to a VIP or referral programme
Loyal Customers	Recency ≥ 3 and frequency-monetary ≥ 3	Offer a loyalty tier upgrade or bundle
Potential Loyalists	Recency ≥ 3	Send a second-purchase incentive
At-Risk Customers	Frequency-monetary ≥ 3	Send a personalised win-back offer
Lost Customers	Everything else	Send a low-cost recovery offer and feedback survey

5.4.3 Health Score

The health score is a number from 0 to 100 built from four factors, each scaled to 0–100. **Purchase frequency** is the rank of the customer's orders per active month among all customers. **Spending behaviour** is the rank of their revenue over the last 12 months. **Engagement** is the average of any engagement scores supplied in the file. **Activity trend** compares the days since the last order with the customer's usual gap between orders: within the usual gap scores 100, and three times the usual gap or more scores 0. The weights are 30%, 30%, 20% and 20%. If a customer has no engagement value, the engagement factor is left out and the other weights are scaled to 37.5%, 37.5% and 25%, so that a missing value is never treated as a zero. The score is placed in a band: Excellent (90 or more), Healthy (75), Watch (55), At risk (30) or Critical (below 30).

5.4.4 Inactivity Level

Where no churn model is available, a rule-based level is shown. The ratio of the days since the last order to the customer's usual gap (the median number of days between distinct order dates, or 90 days if there is only one order) is classified as Low (up to 1.0), Medium (up to 1.5), High (up to 2.5) or Critical (above 2.5). The page labels this as behaviour, not as a prediction.

5.4.5 Key Performance Indicators and Periods

The overview compares a chosen period with the equally long period before it. Revenue is the sum of order values in the period. Retention rate is the percentage of customers who ordered in the

previous period and ordered again in the current one. Total customers and average CLV are measured at the end of the period. A comparison is shown only if the data covers the earlier period; otherwise the card says there is not enough history. By default the period is the 90 days ending on the as-of date, and the user may choose any period of up to 1,095 days.

5.4.6 Growth Score

The growth score is defined by this project and is not an industry standard. It is the weighted average of four measured inputs: revenue growth against the previous period (35%), retention rate (30%), average health score (20%) and customer growth (15%). Revenue growth from −50% to +50% and customer growth from −20% to +20% are mapped to 0–100, and the other two inputs are already on that scale. If an input cannot be measured, it is left out and the remaining weights are scaled up. At least three inputs are needed, otherwise the score is unavailable. The overview shows the inputs, their values and weights.

5.4.7 Cohort Retention

Customers are grouped by the month of their first order. For each group and each later month, the table shows the percentage of the group who placed at least one order in that month. Month 0 is always 100%. The month that is still in progress is not compared, because it would understate retention. The most recent 12 cohorts and 12 months are shown, and the average row weights each cohort by its size. Cohorts under 20 customers are marked, because their percentages vary a lot.

5.4.8 Prediction Models

Three predictions are defined relative to the as-of date. **Churn** means that a customer places no order in the next 90 days. **Predicted CLV** is the revenue to date plus the predicted spending over the next 180 days. The **revenue forecast** extends the monthly revenue for three months.

Training and testing

For churn and spending, features are calculated for each customer using only orders on or before a cutoff date, so no future information is used. There are twelve features: days since the last order, number of orders, orders in the last 90 and 180 days, total spend, spend in the last 90 and 180 days, average order value, typical gap, the ratio of recency to typical gap, days since the first order, and the engagement average (left missing if absent). The label is taken from the period after the cutoff. Training uses the cutoff 180 days before the as-of date, and testing uses the cutoff 90 days before it, so the test period is a later period the model has not seen. The estimator is a histogram gradient boosting model with 150 iterations, a learning rate of 0.08, trees of depth 3 and at least 20 customers per leaf. Features that are constant or entirely empty are dropped before training.

Quality gates

A model is shown only if it passes the gates in Table 5.4. If it fails, the model reports “unavailable” with a message that states the shortfall in words.

Table 5.4: Quality gates for each prediction

Prediction	Gate
Churn	At least 270 days of orders; at least 200 customers with history at the training cutoff; at least 30 customers of each outcome in both the training and the test period; test AUC at least 0.65 and higher than the AUC of the rule “the longer the silence, the higher the risk”.
Predicted CLV	At least 450 days of orders; at least 200 customers at the training cutoff; the mean absolute error on the test period must be lower than that of the baseline “each customer repeats the last 180 days”.
Revenue forecast	At least 12 complete months of revenue; a backtest of the trend forecast on past months must be more accurate than repeating the recent average. The forecast is a linear trend over the last 12 complete months with an 80% prediction range.

Final models and caching

When a model passes, a final model is trained on both periods and scores every customer as of the as-of date. The results are cached together with a fingerprint of the data, and the models are retrained automatically on the first request after the data changes. A model that raises an unexpected error is reported as unavailable and does not break the page.

5.4.9 Recommendations

Each segment has one fixed rule that produces a recommendation with a title, a detail text and a next action, filled with the audience size, the average days since the last order and the segment’s revenue in the last 12 months. The impact is High if the segment holds at least 20% of that revenue, Medium for at least 8%, and Low otherwise. The recommendations deliberately contain no estimated revenue gain, since none can be derived from the data.

5.4.10 Assistant Grounding

The assistant never sees the database. The API builds a summary of the computed figures: KPIs, segments, risk distribution, model status and the metrics of any model that passed. The summary excludes customer names and email addresses. The instructions given to the model require it to answer only from the summary, to call predictions estimates and mention their validation, to say so when the data does not contain the answer, and to list the unavailable items instead of estimating them. If no data has been imported, a fixed reply is returned without calling the provider.

5.5 Security Methodology

Security was handled by listing the main threats to a site that holds customer data and choosing a control for each. Table 5.5 summarises the result. The controls are tested by the suites described in Chapter 7.

Table 5.5: Threats considered and the controls used

Threat	Control
Stolen password database	scrypt hashing with a random salt for passwords [10, 11]; verification codes and session tokens stored only as hashes.

Threat	Control
Guessing passwords	A lock of 15 minutes after 8 wrong attempts per email address. The counter is held in the memory of the server process, so on a hosting platform that runs several instances at once each instance counts separately.
Guessing verification codes	Five wrong tries per code, counted in the database; one new code per minute and five per hour for each address.
Session theft and cross-site requests	HttpOnly, SameSite=Lax cookies, Secure in production; writes from origins that are not on the allow-list are refused.
Account enumeration	One generic message for a wrong email or password; the resend endpoint always answers the same way.
Leaking customer data in errors	Import errors describe the problem without the value. The assistant summary has no names or emails.
Direct access to the hosted database tables	Row-level security is enabled on every table, so the public REST interface of the hosting provider can read nothing [14]. The API connects as the table owner.
Leaked keys	Provider keys and passwords are read from environment settings, never sent to the browser, and excluded from the repository.

Chapter 6

DETAILED DESIGN AND ARCHITECTURE

6.1 System Architecture

CustomerIQ AI is a three-part client-server system: a website, an API and a database, with a small number of external services. The website is the only part the user sees. The API owns all business rules and is the only part that reads or writes the database. This chapter first gives the high-level view and then describes each component and its design diagrams.

6.1.1 Architecture Design Approach

The architecture follows three principles. **Separation of presentation and logic**: the website contains no analytics, and every number it shows comes from the API, which makes it impossible for the interface to quietly invent a figure. **API first**: every feature is an endpoint with a typed request and response, so it can be tested without a browser. **Compute from source data**: analytics are derived from the order table at request time instead of being stored, which keeps them consistent with the data.

Alternatives were considered and rejected. Putting the analytics inside Next.js route handlers would have kept a single project but would have made the Python machine-learning libraries unavailable. Pre-computing the analytics into tables would have made responses faster, but it adds the risk of stale values and a refresh step after every import. Storing sessions as signed tokens in the browser would have removed a database lookup, but it would not allow a session to be revoked.

6.1.2 Architecture Design

Figure 6.1 shows the deployment architecture. The browser requests pages and API calls from the website project. The website forwards `/api/v1` requests, and calls from server-rendered pages, to the API project together with the session cookie. The API connects to PostgreSQL over an encrypted connection. Three optional external services are shown with dashed lines: a public exchange-rate service called by the browser, an AI provider called by the API for the assistant, and an email route called by the API when it sends a verification code. When used on one computer, the same parts run locally: the API on port 8000, the website on port 3001 and a local PostgreSQL server.

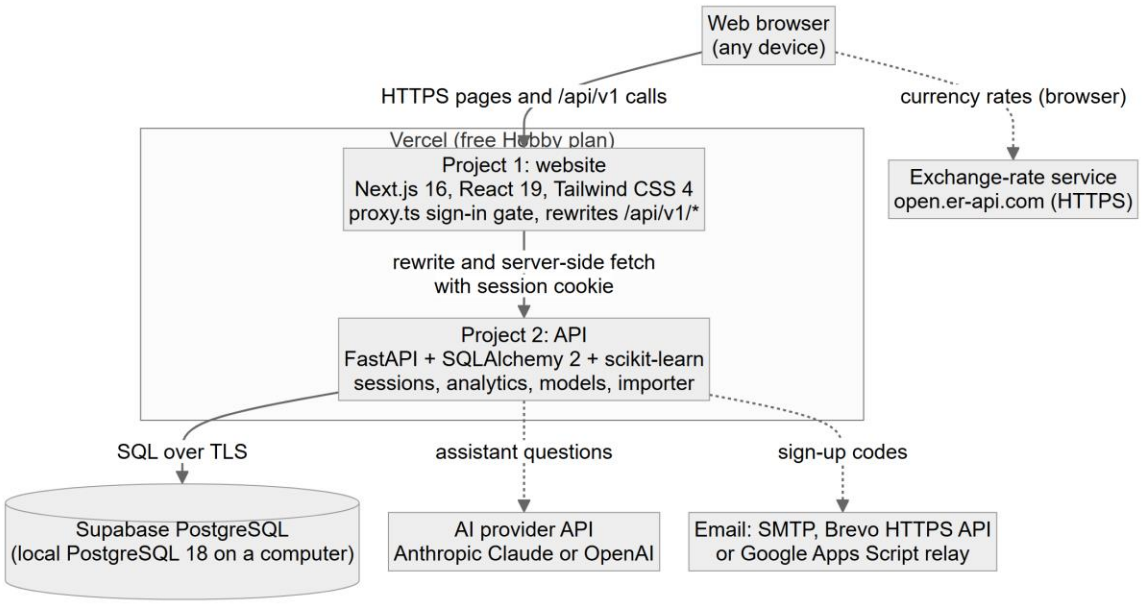


Figure 6.1: Deployment architecture of CustomerIQ AI

Table 6.1: Subsystems and their responsibilities

Subsystem	Responsibility
Website (Next.js)	Pages, charts, import page, calendar, currency and language choices, sign-in gate, forwarding of API calls.
API (FastAPI)	Request validation, sign-in and sessions, import, analytics, predictions, assistant, error format.
Database (PostgreSQL)	Customers, orders, import records, users and sessions. Schema changes are recorded as migrations.
Prediction cache	In-memory results of the last model training, with a fingerprint of the data they were trained on.
External services	Exchange rates, AI answers and email delivery. The system works without any of them, with the related feature switched off.

6.1.3 Subsystem Architecture

Figure 6.2 decomposes the website and the API into components. In the website, `proxy.ts` is the first thing to see a request and redirects visitors without a session cookie. Pages are built from components, and a small library of helpers holds the API clients, the types, the currency and language logic, and the date and CSV utilities. In the API, routers receive requests and call services, and the services use the models and the database session. The prediction service depends on the analytics service for its input, and the assistant service depends on both.

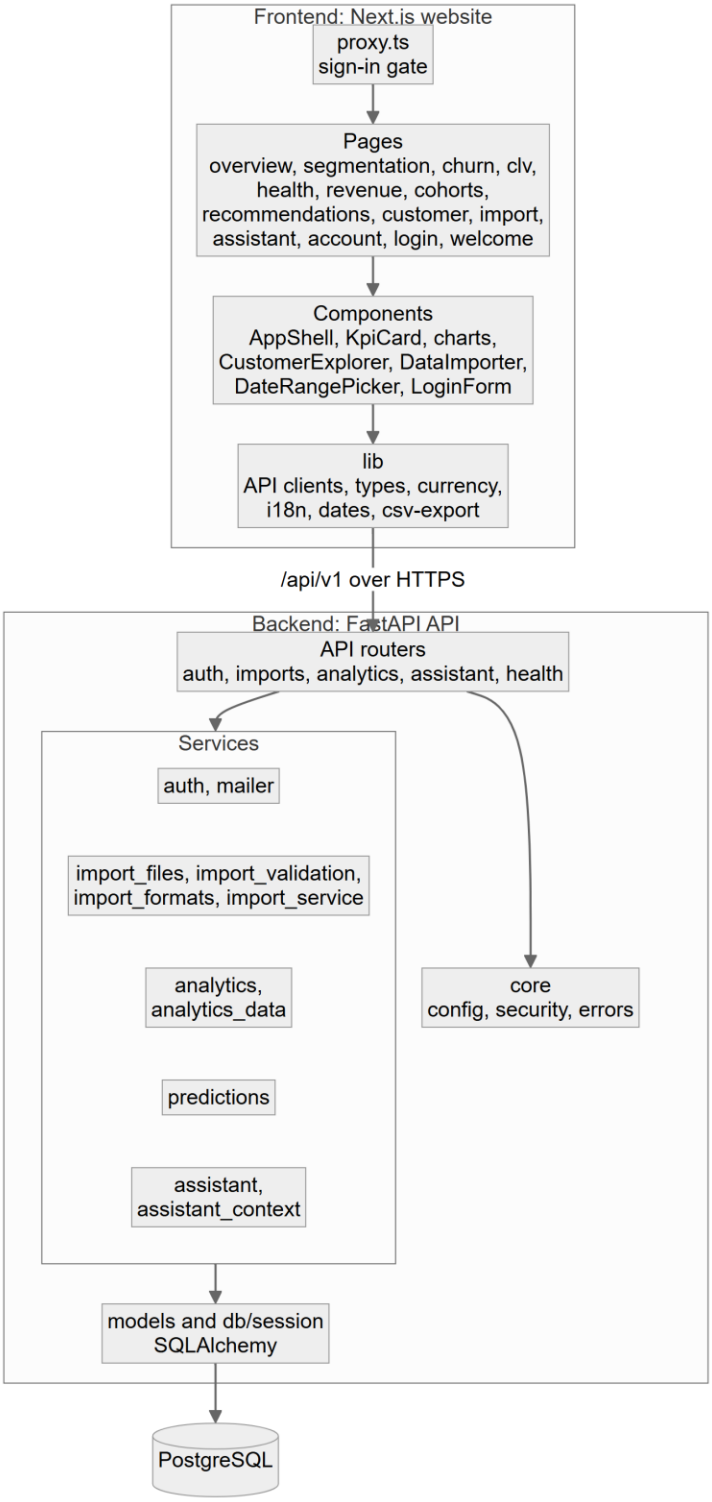


Figure 6.2: Component diagram of the website and the API

Figure 6.3 shows the data flow at the highest level. The user supplies order files, sign-in details, a period and questions, and receives dashboards, predictions, downloads and answers. The system

sends a verification message to the email service and a data summary to the AI provider. Figure 6.4 refines this into six processes and four data stores.

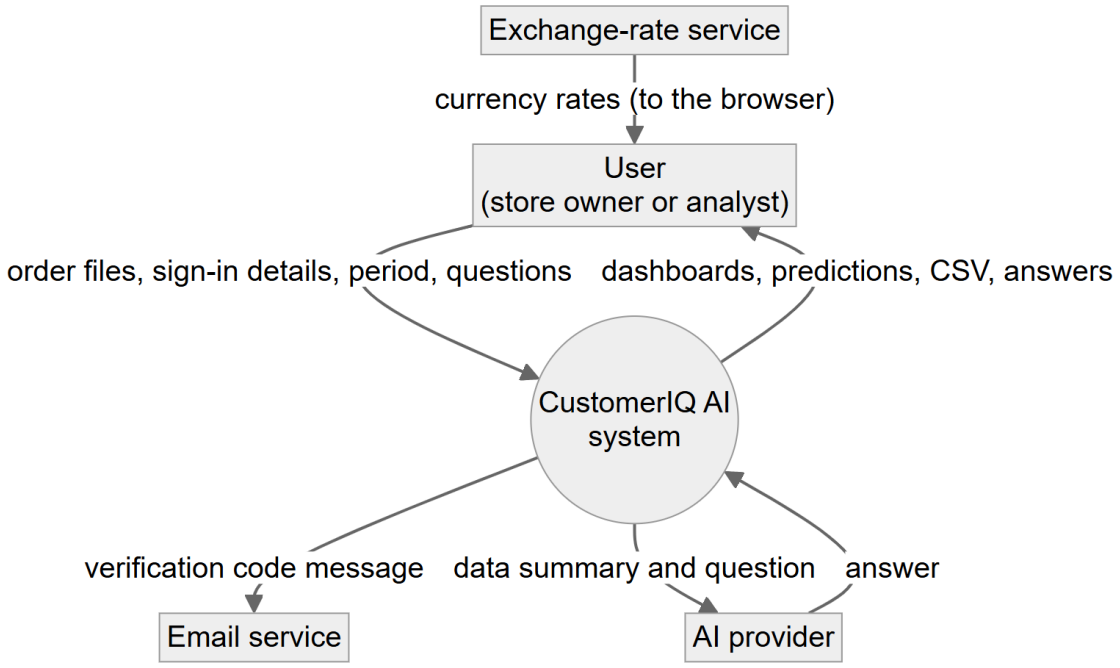


Figure 6.3: Data flow diagram, level 0 (context)

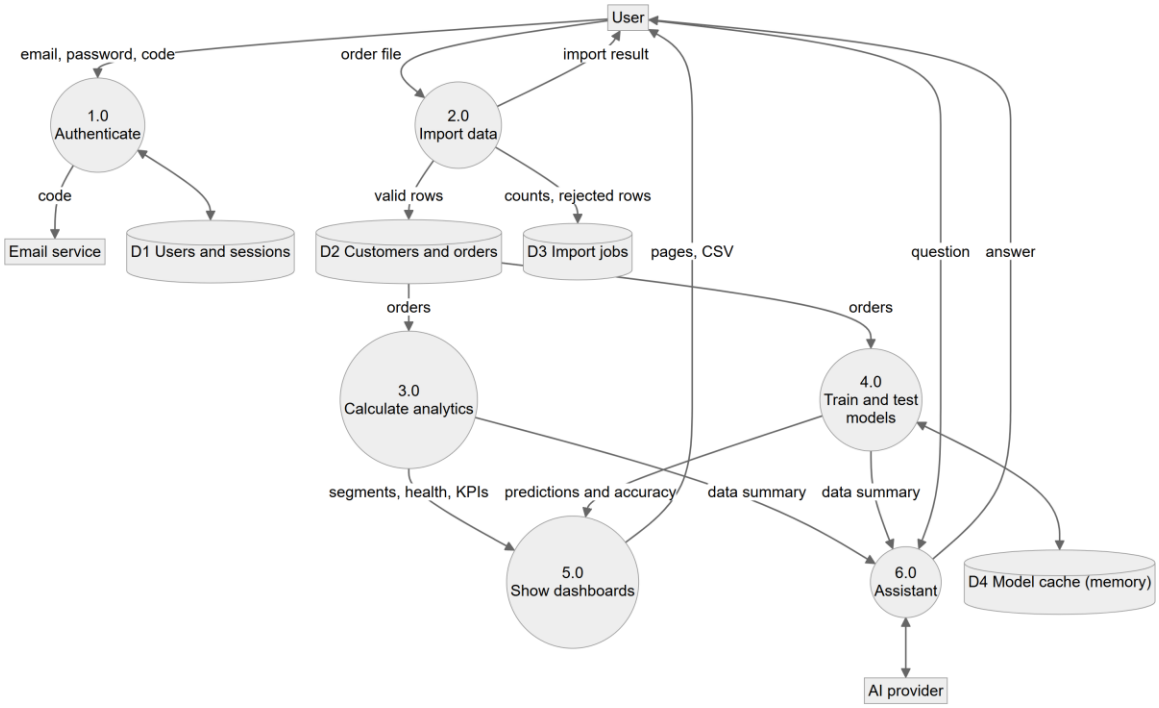


Figure 6.4: Data flow diagram, level 1

6.2 Detailed System Design

The system is described through its six main components. Sections 6.2.1 to 6.2.9 give, for each component, the attributes that the template asks for. Section 6.2.10 then presents the diagrams.

6.2.1 Classification

Table 6.2: Classification of the components

Component	Description
Import module	Back-end subsystem: import_files, import_validation, import_formats, import_service and the imports router.
Analytics engine	Back-end subsystem: analytics, analytics_data and the analytics router.
Prediction engine	Back-end module: predictions.
Authentication and email	Back-end subsystem: the auth router, the auth and mailer services, security helpers and the sign-in dependency.
Assistant	Back-end module: assistant, assistant_context and the assistant router.
Web front end	Next.js application: app, components, lib and proxy.ts.

6.2.2 Definition

Table 6.3: Definition of the components

Component	Description
Import module	Turns an uploaded file into validated customers and orders and records the outcome of the import.
Analytics engine	Calculates segments, health scores, KPIs, cohorts, revenue drivers and recommendations from the stored orders.
Prediction engine	Trains and tests the models, decides whether their predictions may be shown, and applies them.
Authentication and email	Creates and protects accounts, manages sessions, confirms email addresses and sends codes.
Assistant	Answers questions from a summary of the results that contains no personal details.
Web front end	Presents the results, collects input and keeps demonstration data clearly separate from live data.

6.2.3 Responsibilities

Table 6.4: Responsibilities of the components

Component	Description
Import module	Detect the file type and layout; read CSV and XLSX; convert Shopify exports; validate every row; save or update orders; store the import record.
Analytics engine	Build the snapshot of all customers; score RFM and health; compute KPIs for a period; cohorts; revenue drivers; recommendations; executive summary.
Prediction engine	Build features without look-ahead; train and test; apply the quality gates; cache results by data fingerprint; produce the forecast.
Authentication and email	Hash passwords; create and end sessions; limit sign-in attempts; issue, check and limit verification codes; send email through the chosen route.
Assistant	Build the data summary; call the provider; limit the history; translate provider errors into clear messages.

Component	Description
Web front end	Render pages and charts; calendar; currency and language; search and CSV export; forward cookies and API calls.

6.2.4 Constraints

Table 6.5: Constraints of the components

Component	Description
Import module	At most 10 MB and 50,000 data rows per file; 4.5 MB per request on the free hosting tier; UTF-8 or Windows-1252 text.
Analytics engine	The snapshot is built in memory for each request; segments need at least 20 customers; periods are limited to 1,095 days.
Prediction engine	Needs enough history and class balance (Table 5.4). The cache lives in one server process, so a new process retrains on its first request.
Authentication and email	The sign-in lock counter lives in server memory. Verification needs a working email route. Sessions last 24 hours or 30 days.
Assistant	Needs a provider key and credit. Keeps the last 10 turns; a message is limited to 2,000 characters.
Web front end	Live pages need the API. Exchange rates need an internet connection. Only short texts are translated.

6.2.5 Composition

Table 6.6: Composition of the components

Component	Description
Import module	File reader, layout detector, Shopify converter, row validator, importer and import record.
Analytics engine	OrderFact, CustomerMetrics and Snapshot classes; scoring, aggregation and recommendation functions.
Prediction engine	History builder, feature builder, classifier, regressor, trend forecaster and the cache.
Authentication and email	User and UserSession models; password and token helpers; code issuer and checker; mailer routes (SMTP, Brevo, Apps Script).
Assistant	Context builder, Claude client, OpenAI client and error mapping.
Web front end	Pages, KPI cards, charts, customer explorer, data importer, date range picker, login form, currency menu, language selector.

6.2.6 Uses and Interactions

Table 6.7: Uses and interactions of the components

Component	Description
Import module	Writes customers, orders and import records to the database; used by the imports router.
Analytics engine	Reads orders from the database; used by every dashboard endpoint, the prediction engine and the assistant.
Prediction engine	Takes the analytics snapshot as input; used by the analytics endpoints and the assistant context.
Authentication and email	Reads and writes users and sessions; sends email through an external provider; used by every protected endpoint.
Assistant	Uses the analytics and the prediction results; calls the external AI provider.

Component	Description
Web front end	Calls the API; calls the exchange-rate service; keeps the language and currency choices in browser storage.

6.2.7 Resources

Table 6.8: Resources used by the components

Component	Description
Import module	Memory for the file and its rows; database writes in one transaction per import.
Analytics engine	Memory for all orders of one request; database reads.
Prediction engine	Processor time for training (a few seconds at most on the test data); a lock prevents two trainings at once.
Authentication and email	Database; the quota of the email route; the hashing cost of scrypt (about 32 MiB of memory per hash).
Assistant	Provider credit and network access.
Web front end	Browser storage and network; the server of the website for streamed pages.

6.2.8 Processing

Table 6.9: Processing performed by the components

Component	Description
Import module	Described in Section 5.4.1. A bad row never stops the others.
Analytics engine	Described in Sections 5.4.2 to 5.4.7 and 5.4.9.
Prediction engine	Described in Section 5.4.8. A failing model is reported as unavailable.
Authentication and email	Random 6-digit codes and session tokens; only hashes are stored; limits as in Table 5.5.
Assistant	Described in Section 5.4.10.
Web front end	Server components load data with the visitor’s cookie and stream slow sections; client components handle interaction.

6.2.9 Interface and Exports

Table 6.10: Interfaces of the components

Component	Description
Import module	POST /imports, GET /imports, GET /imports/{id}.
Analytics engine	GET /dashboard/overview, /customers, /customer, /segments, /customers/health, /revenue/intelligence, /cohorts, /recommendations.
Prediction engine	get_predictions(snapshot) returns the three model results and the per-customer values; also exposed by /predictions/churn and /predictions/clv.
Authentication and email	POST /auth/setup, /signup, /verify, /resend, /login, /logout, /change-password; GET /auth/status, /auth/me; the require_user dependency.
Assistant	GET /assistant/status, POST /assistant/chat.
Web front end	The pages listed in Section 4.5.1.

6.2.10 Detailed Subsystem Design

The following diagrams describe the structure and behaviour of the system. They were drawn from the implemented code, and the entities, states and flows shown correspond to real tables, classes and functions.

Use case diagram

Figure 6.5 shows the actors and the use cases of the system. Email and AI providers appear as external actors connected to the use cases that depend on them.

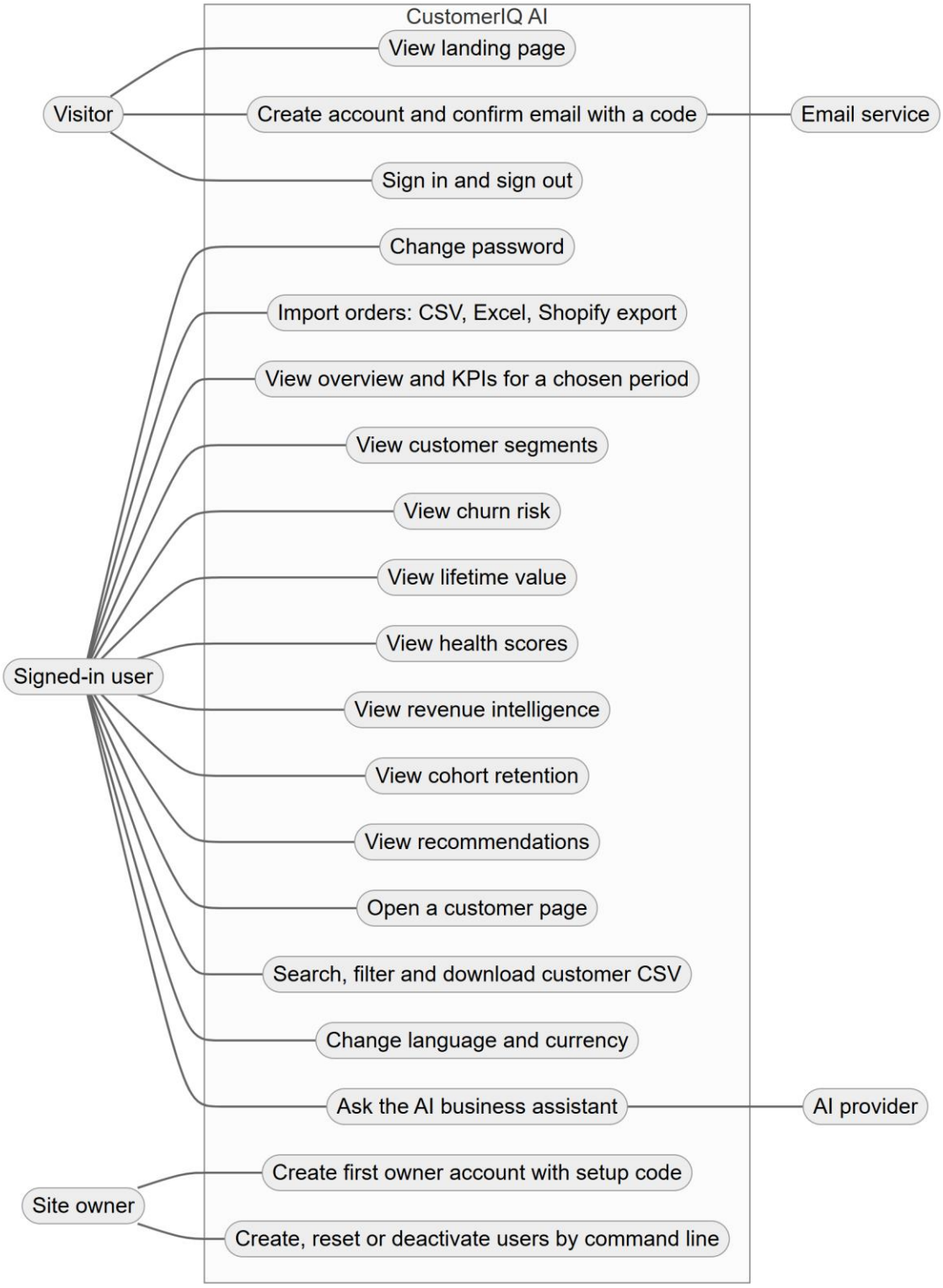


Figure 6.5: Use case diagram

Entity relationship diagram

Figure 6.6 shows the conceptual model. A customer places many orders; an order may have been brought in by an import; a user signs in through many sessions.

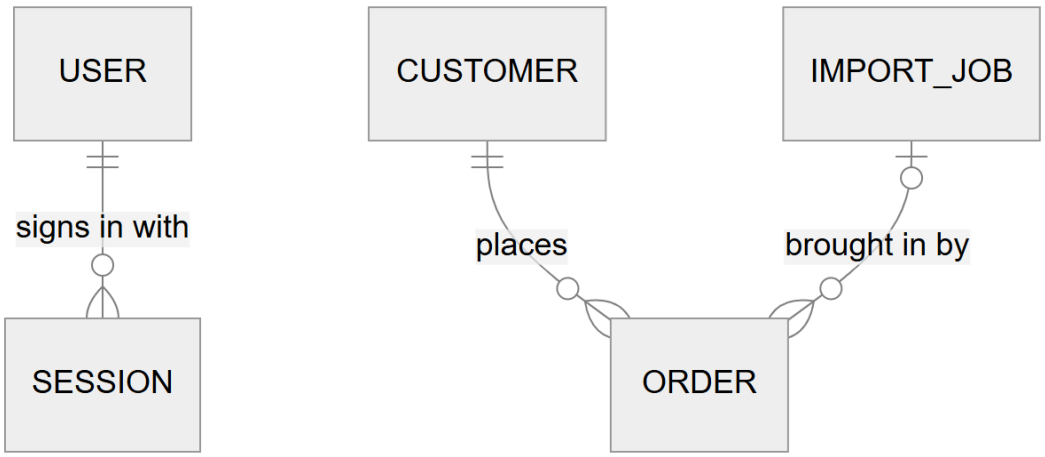


Figure 6.6: Entity relationship diagram

Database diagram

Figure 6.7 shows the physical schema created by the six Alembic migrations. Customer and order identifiers from the imported file are stored as `external_id` and are unique; an order is removed if its customer is removed; and the import link on an order is cleared if the import record is deleted. Order values and engagement scores are protected by check constraints (a value of at least zero, an engagement score between 0 and 100).

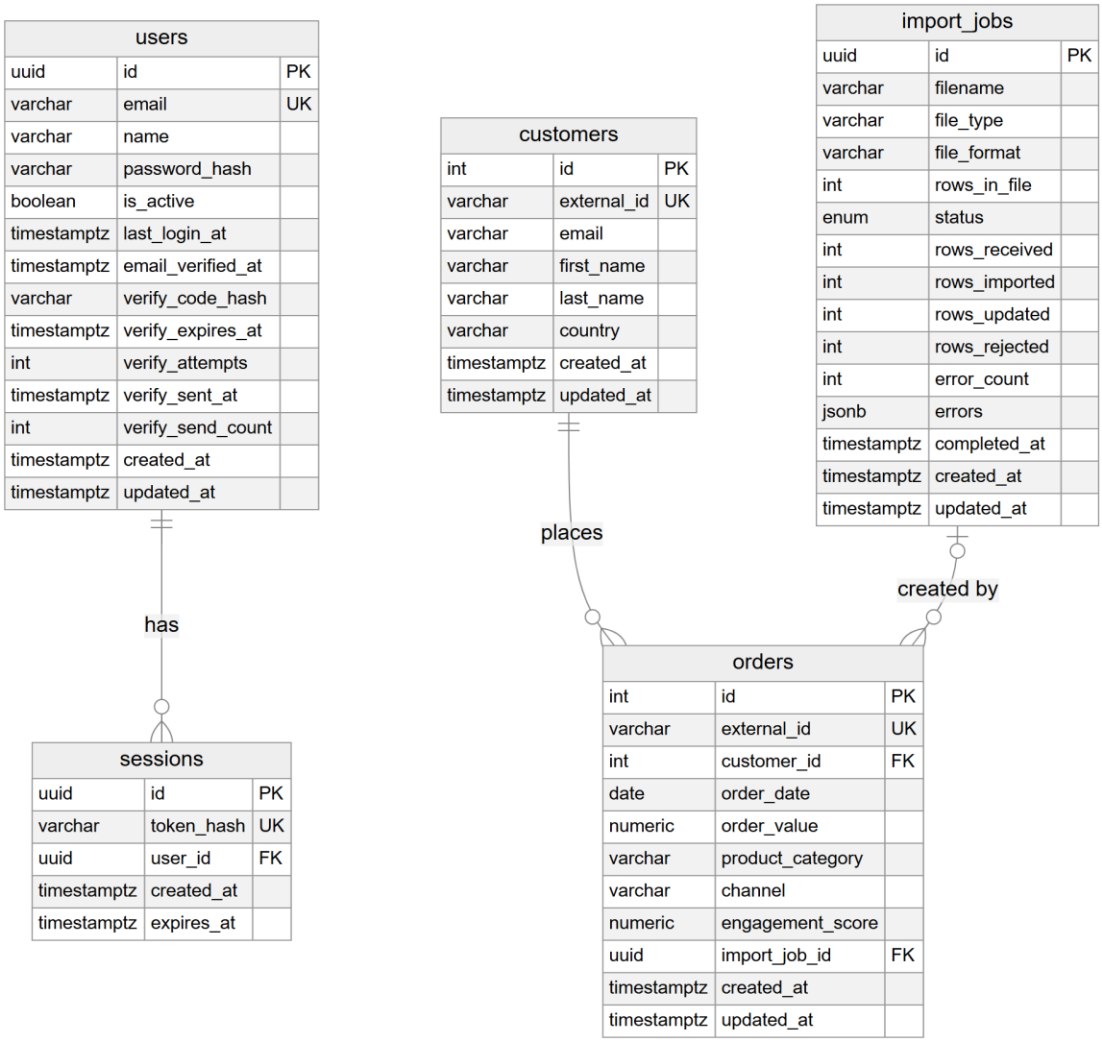


Figure 6.7: Database diagram

Class diagram

Figure 6.8 shows the main classes: the database models, the analytics data classes and the prediction result classes.

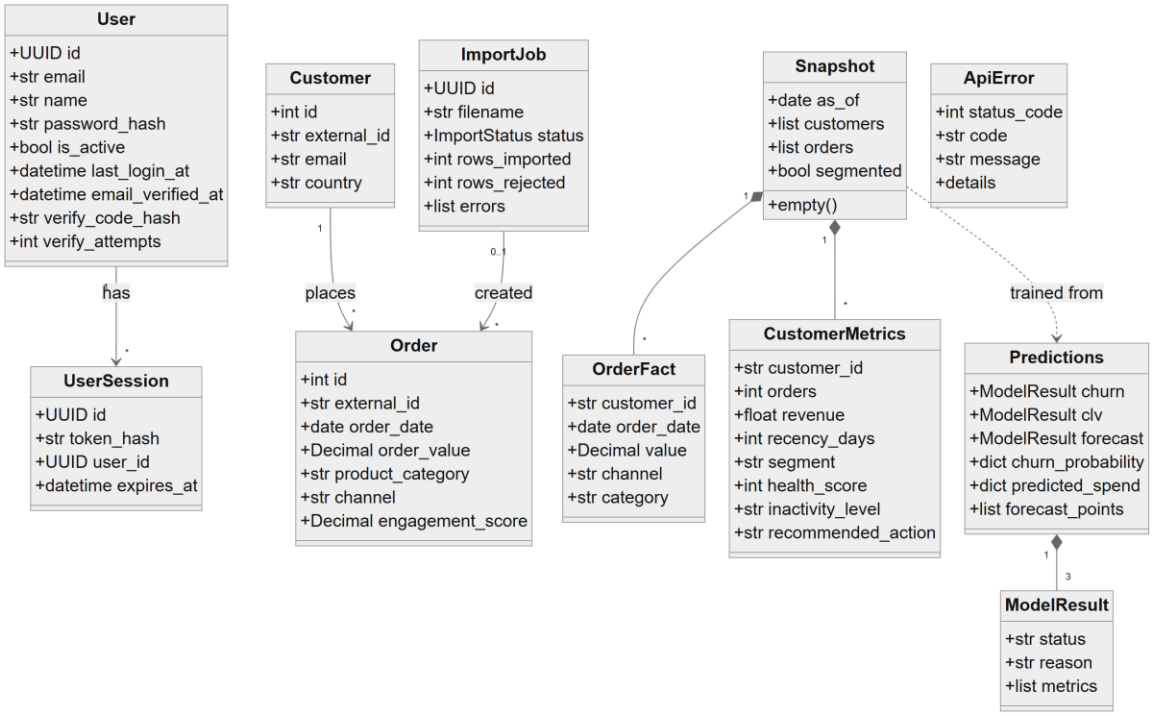


Figure 6.8: Class diagram

Activity diagrams

Figure 6.9 shows the activity of importing a file, including the checks made in the browser before upload and the row-by-row validation on the server. Figure 6.10 shows how a request obtains predictions, including the data fingerprint, the training and the quality gate.

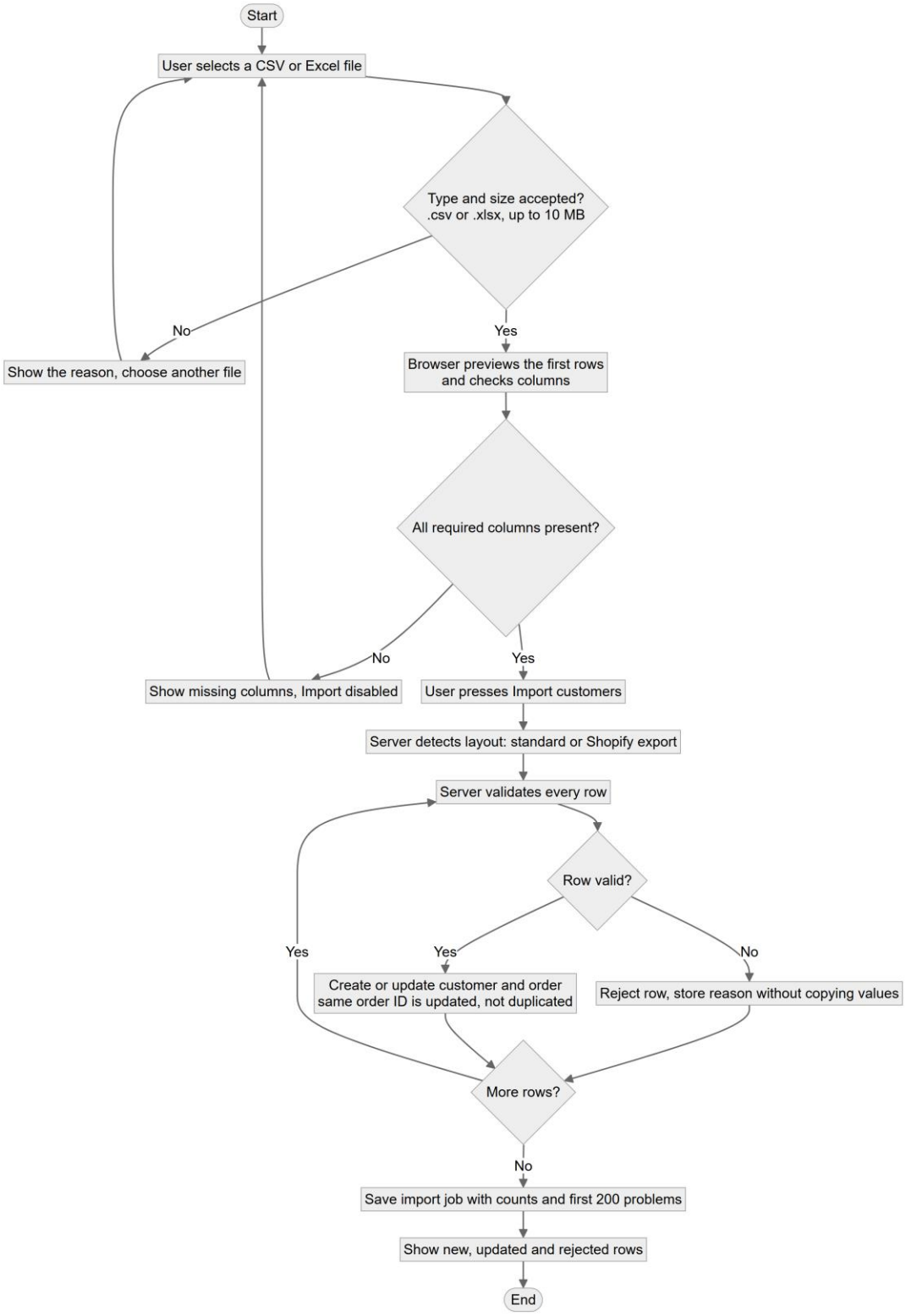


Figure 6.9: Activity diagram: importing orders

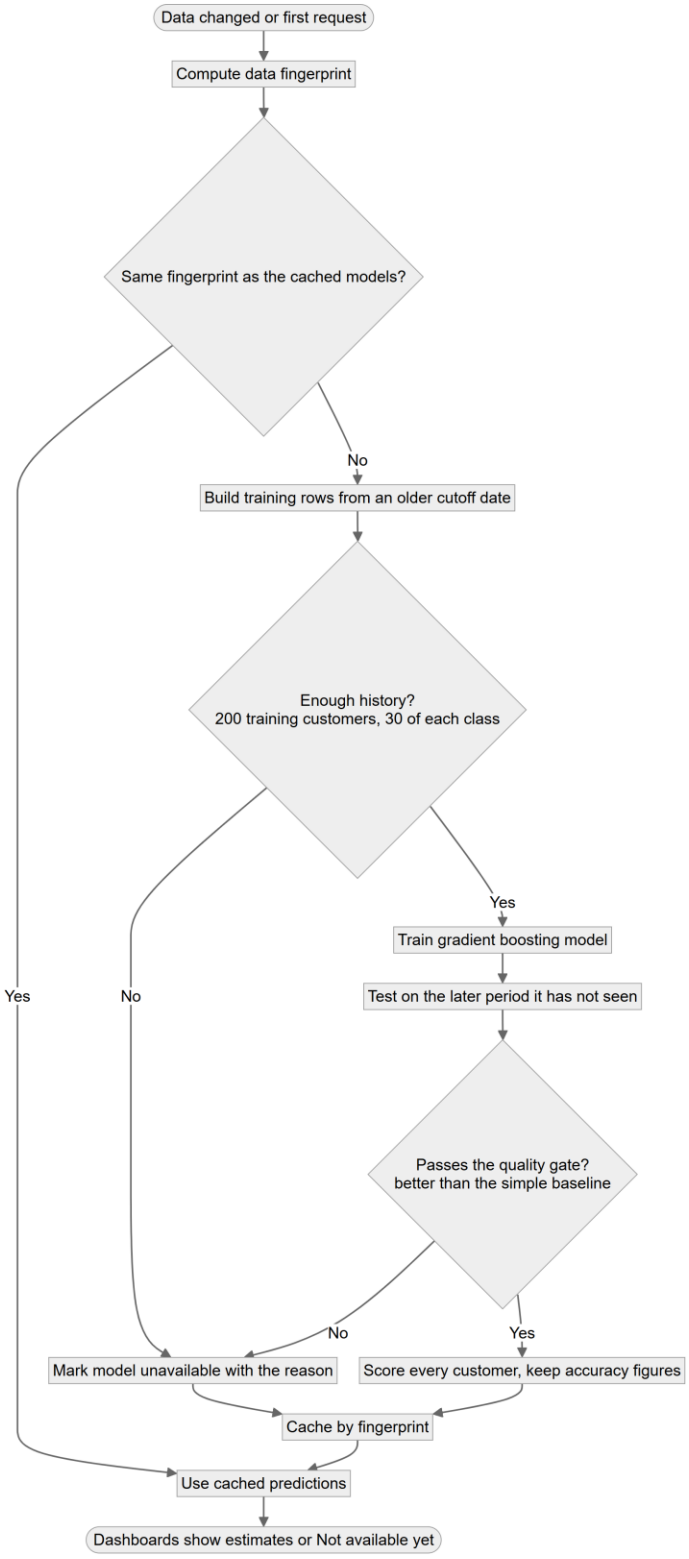


Figure 6.10: Activity diagram: obtaining predictions

Sequence diagrams

Figure 6.11 shows creating an account with an emailed code. Figure 6.12 shows how the overview page is produced for a signed-in user.

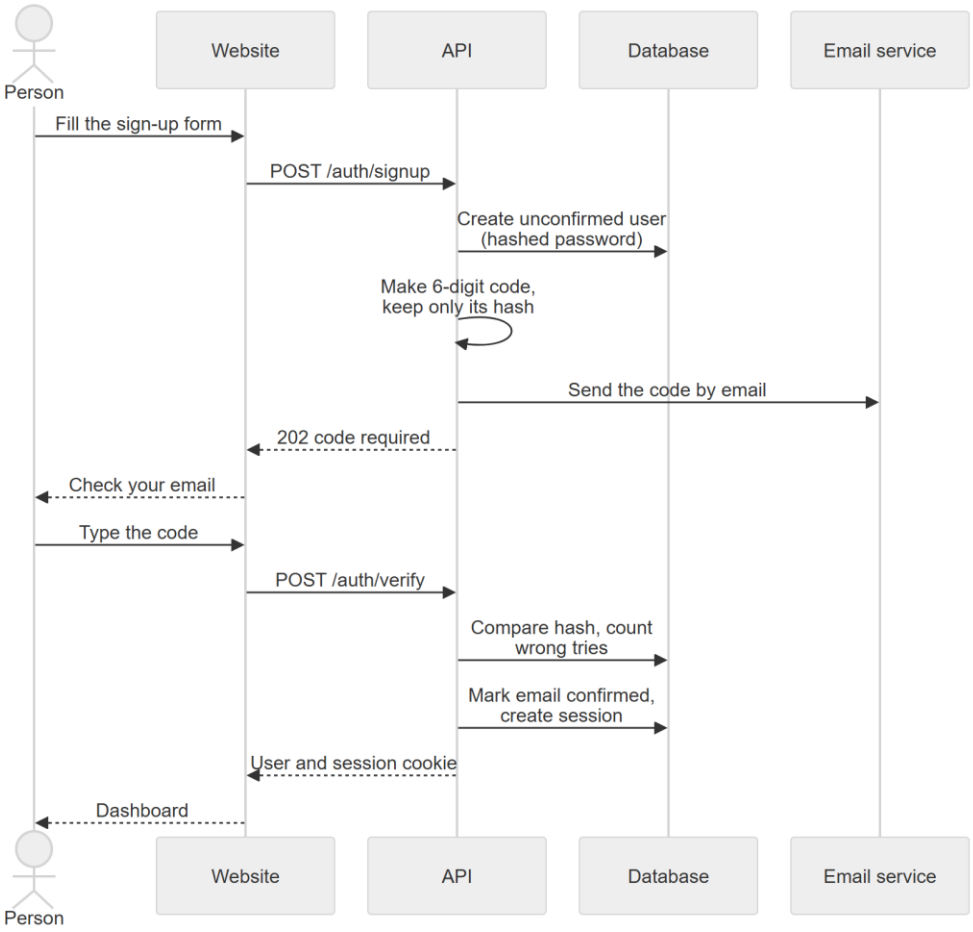


Figure 6.11: Sequence diagram: account creation with email confirmation

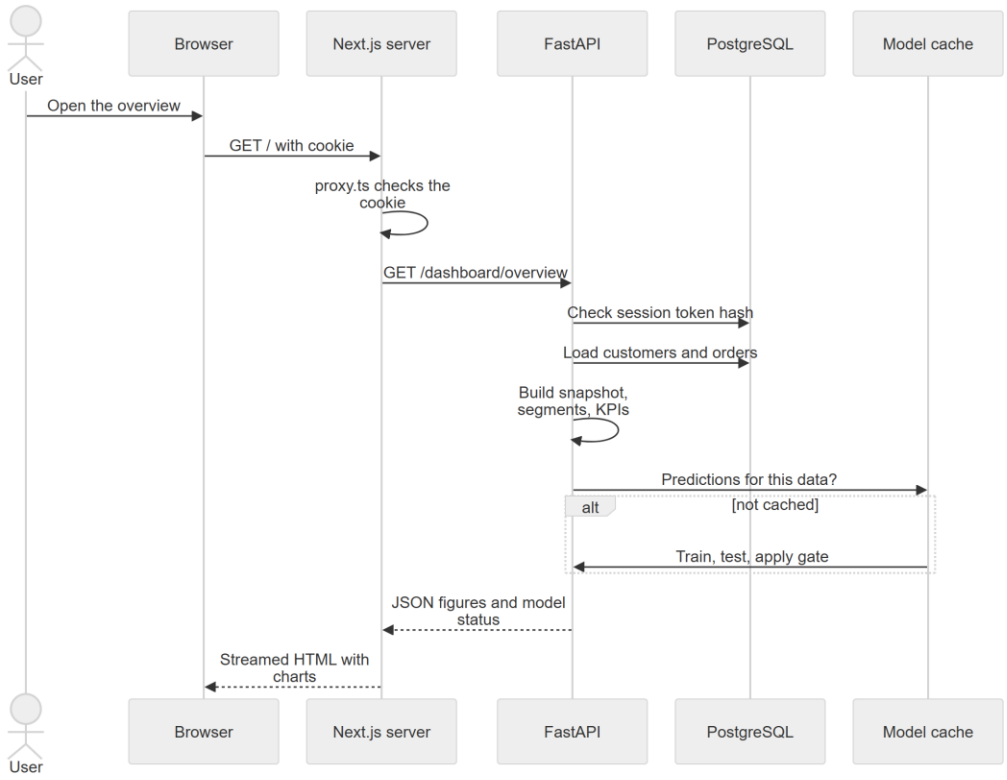


Figure 6.12: Sequence diagram: loading the overview

State machine diagrams

Figures 6.13 to 6.15 show the states of an account, of an import job and of a prediction model. The state names of the import job (pending, processing, completed, failed) are the values stored in the database.

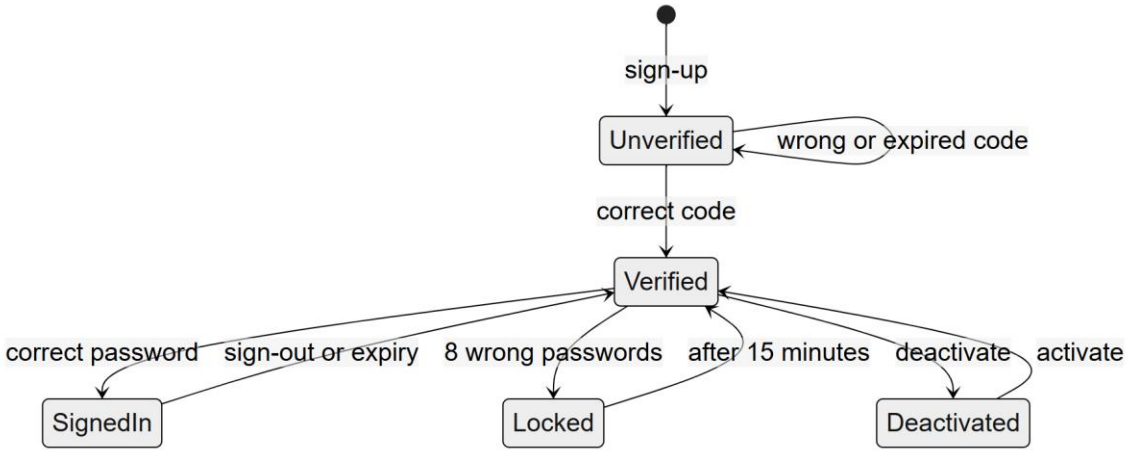


Figure 6.13: State machine diagram: account

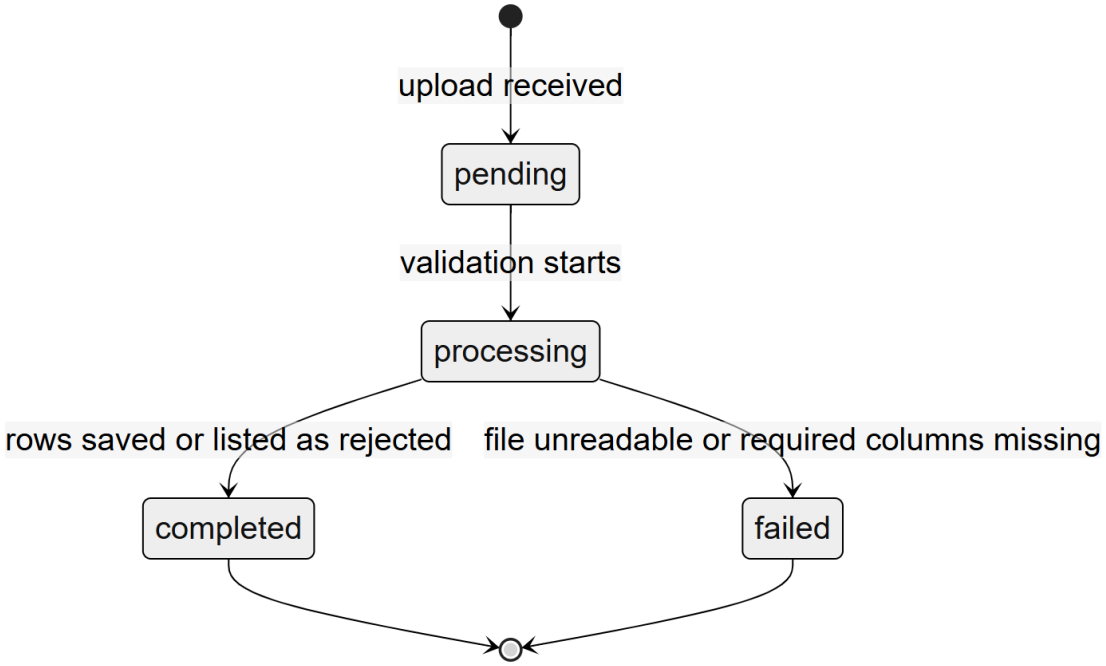


Figure 6.14: State machine diagram: import job

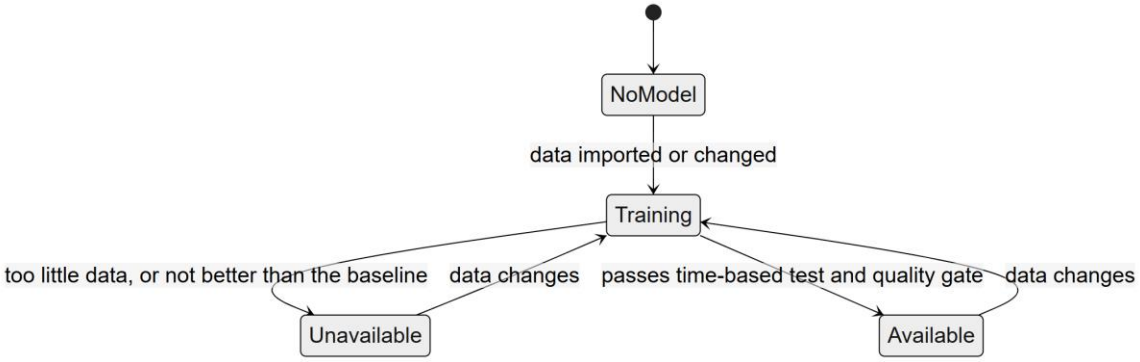


Figure 6.15: State machine diagram: prediction model

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation Overview

The system was implemented in Visual Studio Code with Git for version control, working in the order of the stages listed in Table 5.1. The code base is organised into a `backend` folder for the API, a `frontend` folder for the website, a `docs` folder for the deployment guide, a `data` folder for the converted public dataset (excluded from the repository), and a launcher script that starts both parts on a local computer. Table 7.1 gives the size of each part.

Table 7.1: Size of the code base (counted from the files, excluding libraries)

Part	Language	Files	Lines
API application (backend/app)	Python	40	4,797
Automated tests (backend/tests)	Python	17	2,553
Migrations and environment (backend/alembic)	Python	7	338
Operations scripts (backend/scripts)	Python	5	551
Website (frontend)	TypeScript and TSX	65	8,031

Library versions are pinned in `requirements.txt` for the API and in `package-lock.json` for the website, so that the same versions are installed on every computer and on the hosting platform. The database schema is created and changed only through six Alembic migrations.

7.2 Implementation of Core Functionality

7.2.1 Importing Data

The importer is a pipeline of small functions. A file reader accepts CSV and XLSX files and refuses unsupported types, empty files and files that are only a header. A layout detector decides between the standard layout and a Shopify export. The validator then processes every row and returns either a clean order or a list of problems. Valid rows are written inside one database transaction: a customer is created if the customer ID is new, an order is inserted or updated by its order ID, and the import record stores the counts. Errors name the row, the column and the problem but never repeat the value, so an import record cannot leak a customer’s email or amount. In the browser, the import page reads the first rows of a CSV file locally and shows the detected columns and a preview before anything is sent.

7.2.2 Analytics and Dashboards

The analytics service loads the orders once per request into a snapshot: a list of customers with their measured values, and the list of orders. All dashboards are views of this snapshot. The RFM segment rule is implemented as an ordered series of conditions, so that the first match wins, as in the excerpt below.

```
def segment_for(r: int, fm: int) -> str:
    """First matching rule wins."""
    if r >= 4 and fm >= 4:
        return "Champions"
    if r >= 3 and fm >= 3:
        return "Loyal Customers"
    if r >= 3:
        return "Potential Loyalists"
    if fm >= 3:
        return "At-Risk Customers"
    return "Lost Customers"
```

The overview endpoint accepts a start and end date. The KPI function measures the chosen period and the equally long period before it, ending on the chosen end date, and the growth score and the summary sentence are built from those figures. Cohort retention, revenue drivers with order statistics, and the single-customer view are separate functions on the same snapshot.

7.2.3 Prediction Models and the Quality Gate

The prediction service builds, for every customer, the twelve features at a cutoff date, using only earlier orders. It trains one model on the older cutoff and tests it on the later one, then applies the gates of Table 5.4. The excerpt below shows the two checks that decide whether the churn model is allowed to speak.

```
    if auc < MIN_CHURN_AUC:
        return ModelResult.unavailable(
            f"Not reliable enough yet: accuracy on the test period was {auc:.2f} (AUC), "
            f"below the {MIN_CHURN_AUC:.2f} needed."
        ), {}
    if auc <= baseline_auc:
        return ModelResult.unavailable(
            f"Not reliable enough yet: the model ({auc:.2f} AUC) was no better than a simple
"
            f"'longer since last order = riskier' rule ({baseline_auc:.2f})."
        ), {}
```

The message returned by a failed gate is the message shown to the user, which is why the unavailable pages in Chapter 8 state the measured numbers. Results are cached with a fingerprint of the data, and a lock prevents two requests from training at the same time.

7.2.4 Authentication, Sessions and Email Confirmation

Passwords are hashed with scrypt using Python’s standard library. After a correct sign-in the API creates a random token, stores its SHA-256 hash with an expiry time, and sends the token in an

HttpOnly cookie. For email confirmation, the API creates a six-digit code, stores only its hash, an expiry of 15 minutes, a counter of wrong tries and the time and number of codes sent, and passes the code to the mailer. The mailer supports three routes, chosen by configuration: SMTP (used for local runs), Brevo's HTTPS API, and a small Google Apps Script that sends the message from the owner's Gmail account. A failure is reported to the user with a short reason label such as "login refused" or "unreachable", and never with the text returned by the mail server. A migration marks accounts that existed before email confirmation was added as confirmed, so that nobody was locked out by the change.

7.2.5 AI Business Assistant

The assistant endpoint validates the conversation (at most 20 messages of at most 2,000 characters each), builds the data summary, and calls the configured provider with instructions that restrict it to the summary. Claude is the default provider and OpenAI is an option, chosen by a setting. Provider failures (a wrong key, a missing model, a limit, a timeout, no connection) are mapped to specific messages. The key is read only on the server. In the automated tests the providers are simulated, and no live question was sent to a provider while this report was prepared (Section 8.6).

7.2.6 Website

Pages are server components that call the API with the visitor's cookie and are not cached, so every visit shows current data. Slow sections stream into the page inside Suspense boundaries, with skeletons shown until they arrive. Interactive parts (the customer explorer, the calendar, the importer, the currency and language menus) are client components. The calendar stores the chosen period in the page address, so a link shows the same period. The currency menu keeps the stored values unchanged and multiplies at display time. The CSV export protects against spreadsheet formula injection by prefixing cells that start with `=`, `+`, `-` or `@` and writes a byte-order mark so that Excel reads non-English names correctly. When no API address is configured the website runs in demonstration mode with sample data that carries a visible label.

7.2.7 Deployment

The website and the API are two Vercel projects built from the same repository, with the root directories `frontend` and `backend`. The API reads its database settings from environment variables, and a script, `migrate_remote.py`, applies the migrations to the hosted database after asking for the connection details at the keyboard, so that passwords are never stored in files. The script closes its first connection before the migration and retries a dropped connection, a fix made after a first hosted run failed with a connection error. Both projects were configured for the Singapore region, and the hosted database is a Supabase project with row-level security on every table.

7.3 Testing

7.3.1 Testing Strategy

Testing was done at five levels. **Formula tests** check each calculation on small data sets where the expected result can be worked out by hand. **API tests** call the endpoints through a test client against a real PostgreSQL database, inside a transaction that is rolled back so that the real data is never changed. **Security tests** check hashing, sessions, lockouts and request origins. **Simulated services** replace the mail server, the AI providers and the Brevo and Apps Script endpoints, so that failures can be reproduced. **Browser tests** drive a real browser through the main user flows. The website also has static checks: ESLint, the TypeScript compiler and a production build.

7.3.2 Automated Test Results

The full back-end suite was run on 16 October 2026 with the command `pytest`. All 245 tests passed in about 25 seconds. Table 7.2 shows the number of tests per file.

Table 7.2: Back-end test suite results by file

Test file	Tests	What it covers	Result
test_analytics	40	Formulas: segments, health, KPIs, periods, growth score, cohorts, drivers	40 passed
test_analytics_api	16	Dashboard endpoints with and without data, ranges, customer page	16 passed
test_assistant	31	Context privacy, limits, provider selection, error mapping	31 passed
test_auth	28	Sign-in, sessions, lockout, password change, origins, production mode	28 passed
test_config	9	Settings, database URL, secrets never printed	9 passed
test_convert_uci	2	Conversion rules of the public dataset	2 passed
test_database_integration	2	Database reachable; migrations created the tables	2 passed
test_delete_synthetic	1	Removal of synthetic demo rows only	1 passed
test_email_verification	26	Codes, limits, mail routes, error labels	26 passed
test_health	7	Health check, error format, CORS	7 passed
test_import_parsing	20	File types, limits, headers, encodings, row cleaning	20 passed
test_import_shopify	3	Shopify detection, grouping and end-to-end upload	3 passed
test_imports_api	8	Upload endpoint: partial imports, re-upload, 4xx errors	8 passed
test_predictions	27	Features, labels, gates, forecast, caching, failures	27 passed
test_security	25	Hashing, token storage, password and email rules	25 passed
Total	**245**		**245 passed, 0 failed**

7.3.3 Selected Test Cases

Table 7.3 lists test cases chosen to cover every group of requirements. Each row corresponds to a function in the test suite, whose name is given so that it can be found and re-run. The preconditions are the data set built by the test itself. All cases were executed and the actual result in every case matched the expected result.

Table 7.3: Selected automated test cases

ID	Test (function in the suite)	Steps and data	Expected result	Actual
TC-01	Segment rules, first match wins (test_segment_rules_first_match_wins)	Ten (recency, frequency-monetary) pairs	Each pair gives the segment of Table 5.3	Passed
TC-02	As-of date is the latest order (test_as_of_is_latest_order_date_not_today)	Orders dated 400 and 30 days before a fixed date	As-of date equals the latest order date	Passed
TC-03	Health re-weights without engagement (test_health_reweights_when_engagement_missing)	Two customers with identical orders, only one with engagement values	The first uses the average (70); the second is flagged missing and its other weights are scaled up	Passed
TC-04	Segments need 20 customers (test_segmentation_needs_twenty_customers)	19 customers, then 20	Not segmented, then segmented	Passed
TC-05	Growth score needs three inputs (test_growth_score_needs_at_least_three_inputs)	Only the revenue input measurable; empty data	Score is unavailable in both cases	Passed
TC-06	Period length changes the window (test_kpi_period_length_changes_the_revenue_window)	Orders 5, 60 and 400 days old; periods of 90 and 30 days	Revenue 300 for 90 days; 100 for 30 days	Passed
TC-07	Past period uses data up to its end (test_kpis_for_a_past_period_use_only_data_up_to_its_end)	Period ending before some orders	Later orders and customers are not counted	Passed
TC-08	Cohort retention counts (test_cohort_retention_counts_customers_who_ordered_again_each_month)	Customers with first orders in July and September	50% in month 1 for July; unfinished month left empty	Passed
TC-09	Order statistics per channel (test_revenue_drivers_report_orders_customers_average_and_repeat_rate)	Three Shopify orders by two customers	3 orders, 2 customers, average 100, repeat rate 50%	Passed
TC-10	Bad files refused (test_empty_file_rejected, test_header_only_rejected, test_binary_data_in_csv_rejected)	Empty, header-only and binary files	Each refused with a reason	Passed
TC-11	Missing columns listed (test_missing_required_columns_lists_them)	File without required columns	Error lists the missing columns	Passed
TC-12	Bad rows rejected safely (test_bad_rows_rejected_with_row_numbers_and_no_values)	Rows with bad email, date and value	Row and column named; the value is not repeated	Passed
TC-13	Blank engagement is missing (test_blank_engagement_is_none_not_zero)	Row with an empty engagement cell	Stored as missing, not as 0	Passed
TC-14	Partial import (test_partial_import_saves_valid_rows_and_reports_bad_ones)	File with valid and invalid rows	Valid rows saved; invalid rows reported	Passed

ID	Test (function in the suite)	Steps and data	Expected result	Actual
TC-15	Re-upload updates (test_reupload_updates_existing_orders)	Same order IDs uploaded twice	Orders updated, not duplicated	Passed
TC-16	Missing columns save nothing (test_missing_columns_saves_nothing)	File with a missing required column	Nothing saved; clear error	Passed
TC-17	Shopify export (test_shopify_conversion_groups_line_items_and_skips_canceled_orders)	Export with line items and a cancelled order	Line items grouped; cancelled order skipped	Passed
TC-18	No look-ahead (test_features_ignore_orders_after_the_cutoff)	Orders after the cutoff date	Features do not change	Passed
TC-19	Churn label (test_churn_label_means_no_order_in_the_next_90_days)	Customers with and without later orders	Label 1 only without an order in 90 days	Passed
TC-20	Short history (test_short_history_is_unavailable_with_reason)	60 customers whose orders span about one month	All three models unavailable with “Not enough history”	Passed
TC-21	Train, validate, predict (test_models_train_validate_and_predict)	A synthetic data set of 800 customers with a learnable pattern	Churn model available; AUC at least 0.65 and above the baseline; long-silent customers riskier	Passed
TC-22	Weak forecast rejected (test_forecast_is_rejected_when_no_better_than_repeating_the_average)	Revenue rising for 15 months, then flat for 3	Forecast unavailable: no better than repeating the recent average	Passed
TC-23	Caching and retraining (test_models_are_cached_and_retrained_when_data_changes)	300 synthetic customers, loaded twice, then with one new order	Same result object when unchanged; retrained after the change	Passed
TC-24	Failing model (test_a_failing_model_reports_unavailable_instead_of_crashing)	A churn model made to raise an error (simulated)	Churn reported unavailable (“could not be trained”); the other models are unaffected	Passed
TC-25	Data needs sign-in (test_data_endpoints_need_sign_in_but_health_does_not)	Requests without a cookie	401 for data, 200 for health	Passed
TC-26	Generic failure and lockout (test_wrong_credentials_get_one_generic_message, test_lockout_after_repeated_failures)	Wrong password for known and unknown emails; 8 failures	Same message; then a 15-minute lock	Passed
TC-27	Only hashes stored (test_database_holds_only_hashes)	Create account, read the database	No password or token in clear text	Passed
TC-28	Other websites refused (test_writes_from_other_websites_are_refused)	A write with a foreign Origin header; a read; a write from the allowed origin	Foreign write refused (forbidden_origin); read and allowed write succeed	Passed
TC-29	Production mode (test_production_mode_uses_secure_cookies_and_hides_docs)	Start with production settings	Secure cookie; API docs not served	Passed
TC-30	Password change (test_change_password_signs_out_other_browsers_only)	Two signed-in browsers; change password in one	The other browser is signed out	Passed
TC-31	Email must be confirmed (test_owner_must_confirm_their_email_before_signing_in)	Create the owner with email configured	No session until the code is entered	Passed
TC-32	Wrong codes limited (test_wrong_codes_are_limited_even_if_the_right_one_comes_later)	Five wrong codes, then the right one	Locked: too many attempts	Passed

ID	Test (function in the suite)	Steps and data	Expected result	Actual
TC-33	Expired code (test_expired_code_is_refused)	Move the expiry into the past	Refused: code expired	Passed
TC-34	Resend limits (test_resend_is_rate_limited_and_never_reveals_who_has_an_account)	Resend within a minute; unknown address	No extra email; same answer for all	Passed
TC-35	Mail failure (test_mail_failure_gives_a_friendly_error_without_leaking_details)	A mail server that refuses the login	Short reason; no password or server text	Passed
TC-36	Apps Script route (test_webhook_sends_the_message_and_follows_googles_redirect)	A stand-in server that redirects the POST	Message posted once; redirect followed	Passed
TC-37	Assistant privacy (test_context_never_contains_names_or_emails)	Data with names and emails	No name or email in the summary	Passed
TC-38	Assistant without key or data (test_not_configured_returns_503_without_calling_ai, test_no_data_gives_fixed_reply_without_ai_call)	No key; then no data	503 without a call; fixed reply without a call	Passed
TC-39	Provider errors (test_provider_errors_become_clear_api_errors)	Six simulated provider failures	A specific status and message for each	Passed
TC-40	Secrets never printed (test_password_never_appears_in_str_or_repr)	Print the settings object	No password in the text	Passed
TC-41	Degraded health (test_health_degraded_returns_503)	Database unconfigured, then unreachable	503 with the status named	Passed

7.3.4 Browser Test Cases

The main user flows were run in Google Chrome, controlled by Playwright, against a local production build of the website and a local copy of the API. For the sign-up flow the API sent its emails to a small local mail server that recorded the code, so that the test could read it. The data flows used the public dataset inside a transaction that was rolled back afterwards. The results are in Table 7.4. The test ran on 16 October 2026.

Table 7.4: Browser test cases and observed results

ID	Scenario	Expected result	Observed result
UI-01	A visitor who is not signed in opens the site address	The landing page is shown	Redirected to /welcome; landing page shown
UI-02	First run: the owner fills the account form	The code screen opens and a code is emailed	“Check your email” screen; one code recorded
UI-03	Enter a wrong code	An error is shown; no sign-in	“That code is not correct.”
UI-04	Enter the right code	The owner is signed in	Signed in; empty-state dashboard shown
UI-05	Sign out; open the sign-in page; choose Create an account	Sign-in and sign-up forms are shown	Both shown
UI-06	A second person signs up and confirms the code	Second account is signed in	Signed in
UI-07	Choose a 300-order CSV file	A preview and the Ready to import label	Shown

ID	Scenario	Expected result	Observed result
UI-08	Press Import customers	Import complete with counts	Completed
UI-09	Import a file with two bad rows	No rows saved; four problems listed by row and column	Four problems listed; values not repeated
UI-10	Open every dashboard after importing the public dataset	Pages show figures; unavailable models show reasons	All pages rendered; CLV and forecast showed reasons
UI-11	Open the period calendar, choose This year, Apply	Address gains from and to; figures change	Period 1 Jan to 9 Dec 2011 (343 days); revenue and retention changed
UI-12	Set data currency to GBP and display currency to PKR	Amounts and the rate with its date are shown	1 GBP = 367.3979 PKR with the rate date; amounts converted
UI-13	Switch the language to Urdu and Spanish	Menu and card titles change	Changed; long texts stayed in English
UI-14	Open the page of the customer with most orders	Figures, health factors and 200 latest orders	373 orders; latest 200 listed
UI-15	Open the site on a 390-pixel-wide screen	No horizontal clipping	Landing, sign-in and overview fitted

7.3.5 Static Checks and Build

For the website, ESLint reported no warnings or errors, the TypeScript compiler reported no errors, and `next build` completed and generated 17 pages. The production build is the same one that was deployed.

7.3.6 Limits of the Testing

- The website has no automated unit tests. Its behaviour is covered by the static checks and the browser walk-through of Table 7.4, which is a one-time run, not a permanent suite.
- All data-driven tests used either small hand-made data or one public dataset. No real store data and no users were involved, so nothing can be said about usability with real owners.
- Performance was measured on one computer without concurrent users (Section 8.1). No load test was run.
- Live assistant answers were not evaluated, and email delivery from the hosted site was not working at the time of writing (Sections 8.6 and 8.7).

Chapter 8

RESULTS AND DISCUSSION

This chapter reports what the system produced when it was run on the public Online Retail II dataset, what the browser tests showed, and how far the objectives of Section 1.3 were met. All numbers come from runs made on 16 October 2026 on the development computer, and the dataset's currency is pounds sterling. The interface shows a dollar sign by default, which can be changed in the currency menu, so amounts quoted in this chapter are in pounds unless a figure caption says otherwise.

8.1 Import and Performance Results

The dataset was converted from the original workbook into the import format by the project's conversion script (cancellations and guest checkouts removed, order values summed per invoice) [9]. The resulting file has 36,594 orders from 5,852 customers, between 1 December 2009 and 9 December 2011, and is 3.3 MB in size. It was imported through the API inside a transaction that was rolled back afterwards.

Table 8.1: Result of importing the public dataset

Measure	Value
Rows received	36,594
New orders saved	36,594
Orders updated	0
Rows rejected	0
Status of the import	completed
Customers created	5,852

Response times were measured with the test client on the development computer, which excludes the network between a browser and a server and involves one user only. Table 8.2 shows the results.

Table 8.2: Measured response times on the public dataset (single run, development computer)

Operation	Time (seconds)	Note
Import of 36,594 orders (3.3 MB file)	21.6	Includes validation and database writes
Overview, first request	2.42	Trains the models
Overview, second request	1.16	Models cached

Operation	Time (seconds)	Note
Segments	0.68	
Churn	0.81	
Lifetime value	1.03	
Health scores	0.74	
Revenue intelligence	0.84	
Cohorts	0.76	
Customer list	0.68	
Recommendations	0.70	
One customer page	0.81	373 orders

Every page after the first took between 0.7 and 1.2 seconds. The cost is dominated by loading and processing the orders, which happens at each request because nothing is pre-computed (Section 5.3). Training the three models added about 1.3 seconds to the first request. These figures are adequate for a data set of this size, but they were measured once and say nothing about much larger stores or many simultaneous users. If the number of orders grew by an order of magnitude, the per-request loading would become the first thing to optimise, for example by caching the snapshot.

8.2 Descriptive Analytics Results

Key figures

Table 8.3 shows the figures on the overview for the 90 days ending on 9 December 2011, compared with the previous 90 days.

Table 8.3: Overview figures for the 90 days ending 9 December 2011

Measure	Value	Change on previous 90 days
Total customers	5,852	+11.3%
Revenue	3,389,155.50	+86.1%
Average customer lifetime value (revenue to date)	2,926.34	+12.0%
Retention rate	69.0%	+13.6 percentage points
Predicted churn risk (model estimate)	46.5%	No comparison
Growth score	78 / 100	No comparison

The growth score of 78 follows from its inputs: revenue growth of +86.1% (sub-score 100, weight 35%), retention of 69.0% (69, 30%), an average health score of 51 (51, 20%) and customer growth of +11.3% (78, 15%). The large revenue growth should be read with the dataset’s seasonality in mind. The revenue chart (Section 8.4) shows peaks in the autumn of both years, and the last 90 days of this dataset include the autumn peak while the 90 days before them do not. The system compares equal periods, but it does not adjust for the season.

Segments

Table 8.4: Customer segments on the public dataset (revenue to date)

Segment	Customers	Share	Revenue	Share of revenue	Average CLV
Champions	1,525	26.1%	12,218,135	71.3%	8,012
Loyal Customers	1,251	21.4%	2,443,362	14.3%	1,953
Potential Loyalists	737	12.6%	237,861	1.4%	323
At-Risk Customers	899	15.4%	1,808,881	10.6%	2,012
Lost Customers	1,440	24.6%	416,703	2.4%	289
Total	**5,852**	**100%**	**17,124,942**	**100%**	

Champions are about a quarter of the customers and hold 71% of the revenue to date, which is the kind of concentration that RFM analysis is expected to reveal in retail data. Segment sizes are partly a consequence of the method, because scores are quintiles of the customer base: a segment cannot be very small or very large by accident. The informative part is the revenue column. The At-Risk segment (899 customers) has an average value close to that of the Loyal segment but has not bought recently, and it is the group for which a win-back action has the most revenue to protect. The revenue at risk page counts 360,311 spent by At-Risk customers and 90,277 spent by Lost customers in the last 12 months, 5.3% of the 8,451,543 earned in that year. These are historical amounts, not a forecast of loss.

Health scores

Table 8.5: Distribution of health scores

Band	Score range	Customers	Share
Excellent	90–100	502	8.6%
Healthy	75–89	1,133	19.4%
Watch	55–74	1,472	25.2%
At risk	30–54	978	16.7%
Critical	0–29	1,767	30.2%

Because the dataset has no engagement scores, every customer was scored with the re-weighted formula of Section 5.4.3, and the page says so. About 30% of the customers are in the Critical band. This is probably a consequence of the data: it covers two years of trading, and many customers placed few orders long ago, which gives a low frequency and activity score.

Cohorts and revenue drivers

The cohort table shows that on average 23.6% of a month’s new customers placed another order in the following month, 25.4% in the second month and 21.7% in the third. The percentages are almost flat, with no steep decline, which is typical of a retailer with a stable base of repeat buyers. The newest cohort (December 2011, 28 customers) is small, and the page marks it. For revenue

drivers, the dataset has a single channel and no product categories, so the channel row shows "Online retail" with 17,628 orders from 4,261 customers in the last 12 months, an average order of 479.44 and 64.5% of customers who bought again. By country, the United Kingdom accounts for 82.7% of revenue; the Netherlands and Ireland (EIRE) each account for about 3%, but with only 8 and 3 customers, which shows why the order and customer counts are printed beside each percentage.

8.3 Prediction Results

Table 8.6 reports the outcome of the three models on the public dataset. These are the values the application itself displays, with the same wording.

Table 8.6: Prediction models on the public dataset

Model	Status	Measured on the unseen period
Churn (no order in the next 90 days)	Available	AUC 0.80 against 0.76 for the recency rule; Brier score 0.196; trained on 10,208 customer snapshots; tested on the last 90 days of orders, which were not used for training
Predicted CLV (next 180 days)	Unavailable	Mean absolute error 689, no better than the baseline of repeating the last 180 days (620)
Revenue forecast (3 months)	Unavailable	On past data the trend forecast was off by 56% on average, no better than repeating the recent average (39%)

The churn model passed both gates. An AUC of 0.80 means that, if one customer who did not buy in the test period and one who did are picked at random, the model gave the first a higher risk in about 80% of pairs. The simple rule "the longer since the last order, the higher the risk" achieves 0.76, so the model's advantage is real but modest. The Brier score of 0.196 can be compared with about 0.25, which is what a model that always predicts a base rate of one half would obtain (the exact figure depends on the share of customers who churn, which is close to one half here). This improvement is also modest, and the probabilities should be treated as a ranking aid and not as exact chances.

The other two models were withheld, and this is the behaviour the project aims for. The spending model did worse than the baseline, so showing its numbers would have been misleading. The trend forecast was beaten by repeating the recent average; a plausible reason is that the monthly revenue has a strong yearly pattern that a straight trend line cannot follow, but this was not investigated further. In both cases the pages display the reason with the numbers (Figures in Section 8.4). A tool that returned a forecast anyway would look more complete but would be less truthful.

These results cannot be generalised. They describe one retailer's data, and they are not a promise that the models will pass the gates on another store.

8.4 User Interface Walkthrough

The figures in this section are screenshots of the running application, taken with the public dataset loaded. The signed-in user shown in the header is a test account, and the customers are identified by numbers only.

Landing, sign-up and confirmation

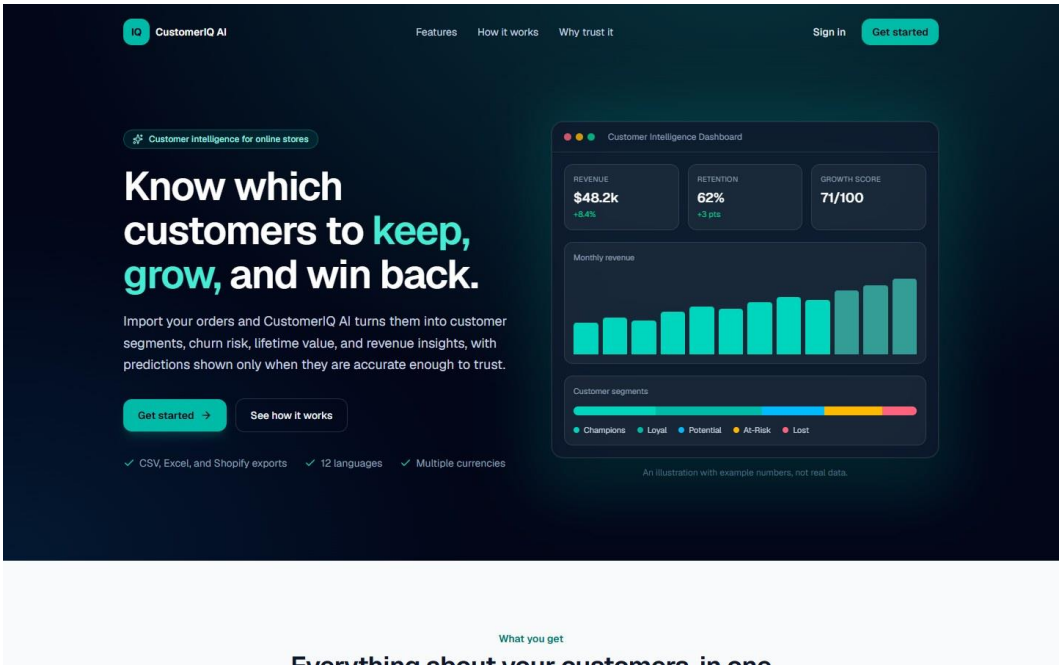


Figure 8.1: Landing page shown to visitors who are not signed in

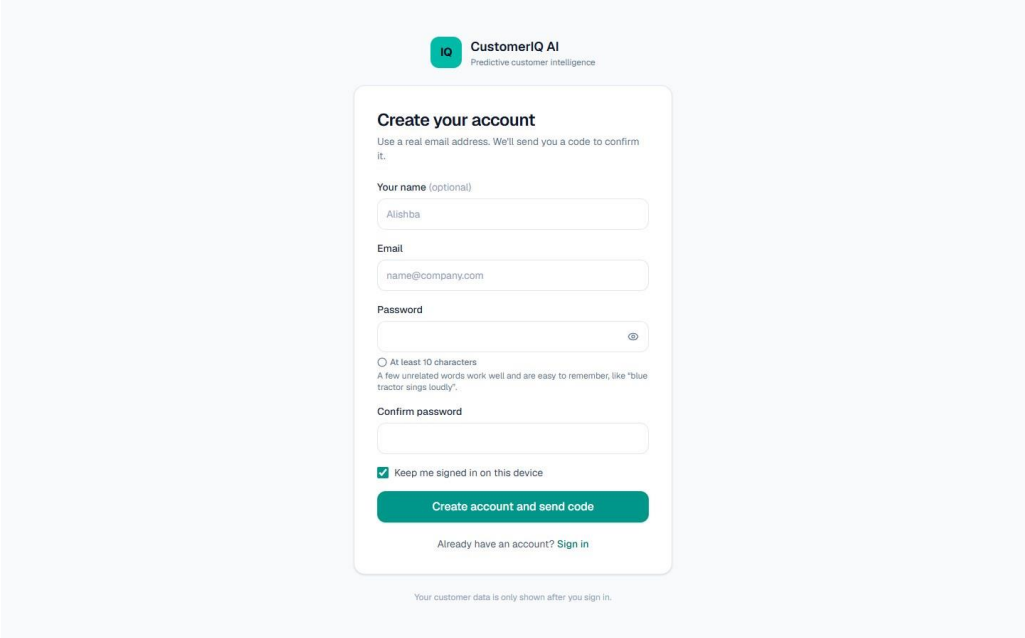


Figure 8.2: Sign-up form (shown when email confirmation is configured)

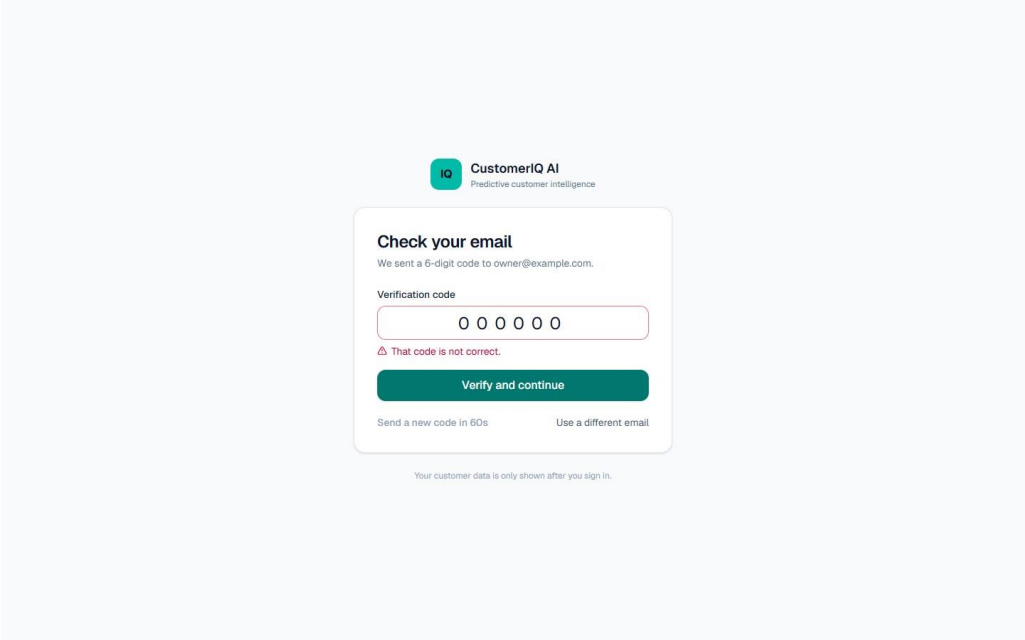


Figure 8.3: Code screen after a wrong code was entered

Overview and period selection

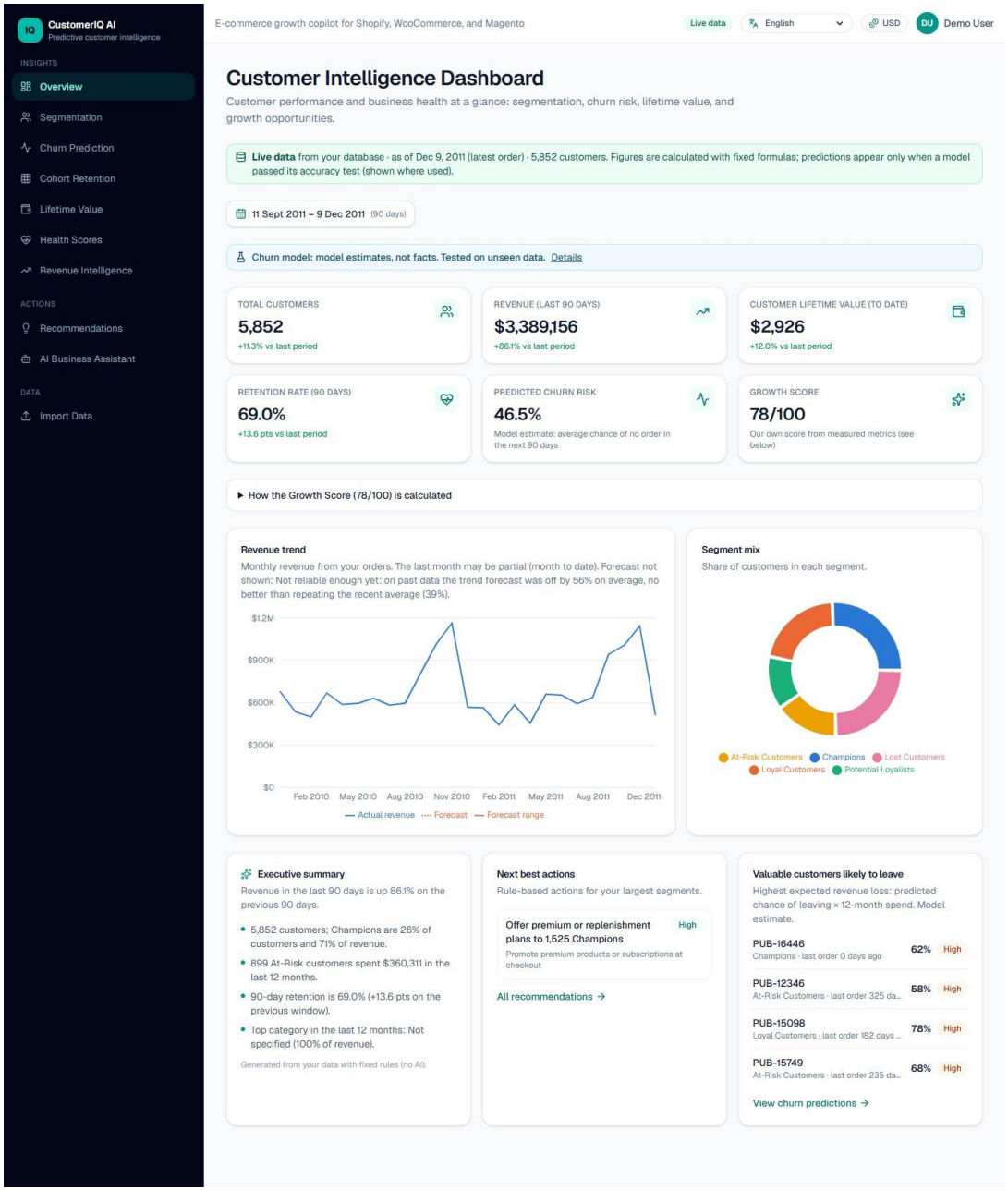


Figure 8.4: Overview dashboard with the key figures, revenue trend and segment mix

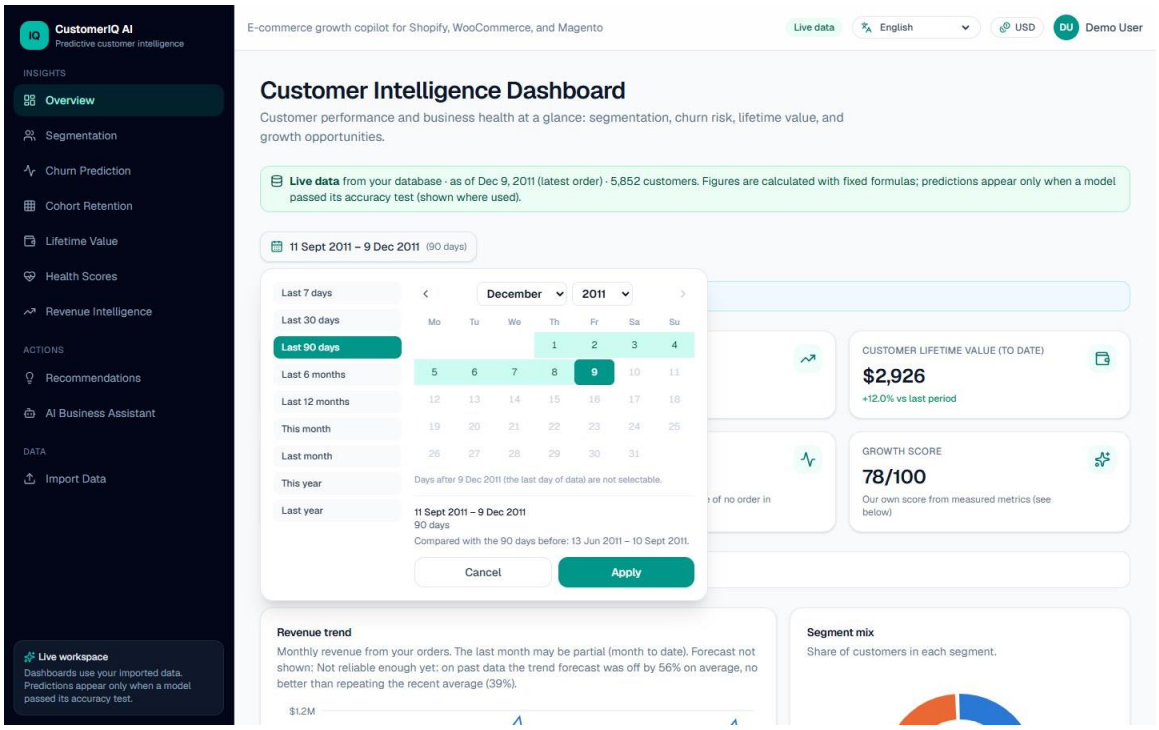


Figure 8.5: Period calendar with quick choices and the comparison period

Segments, churn, value and health

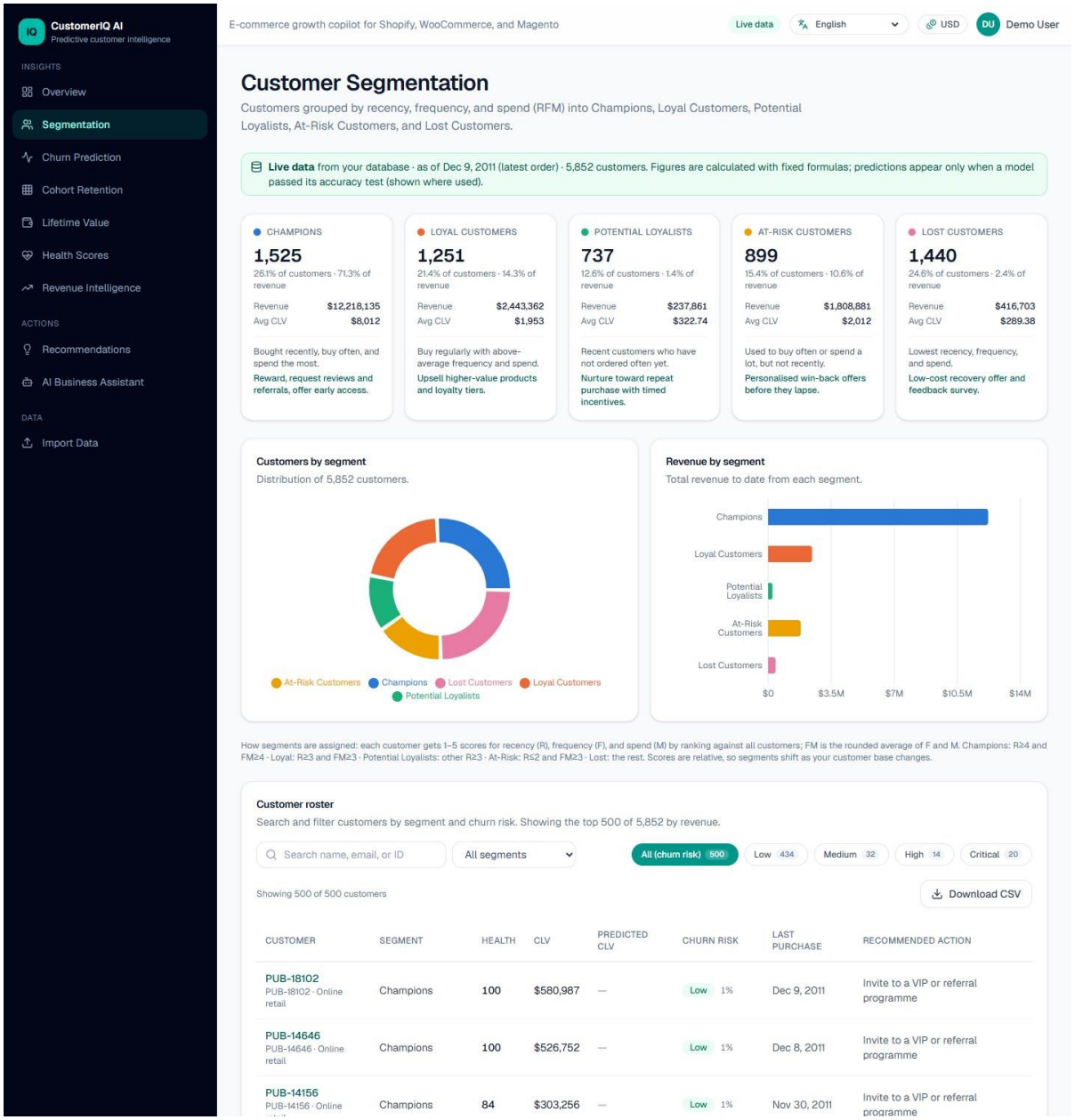


Figure 8.6: Customer segmentation page

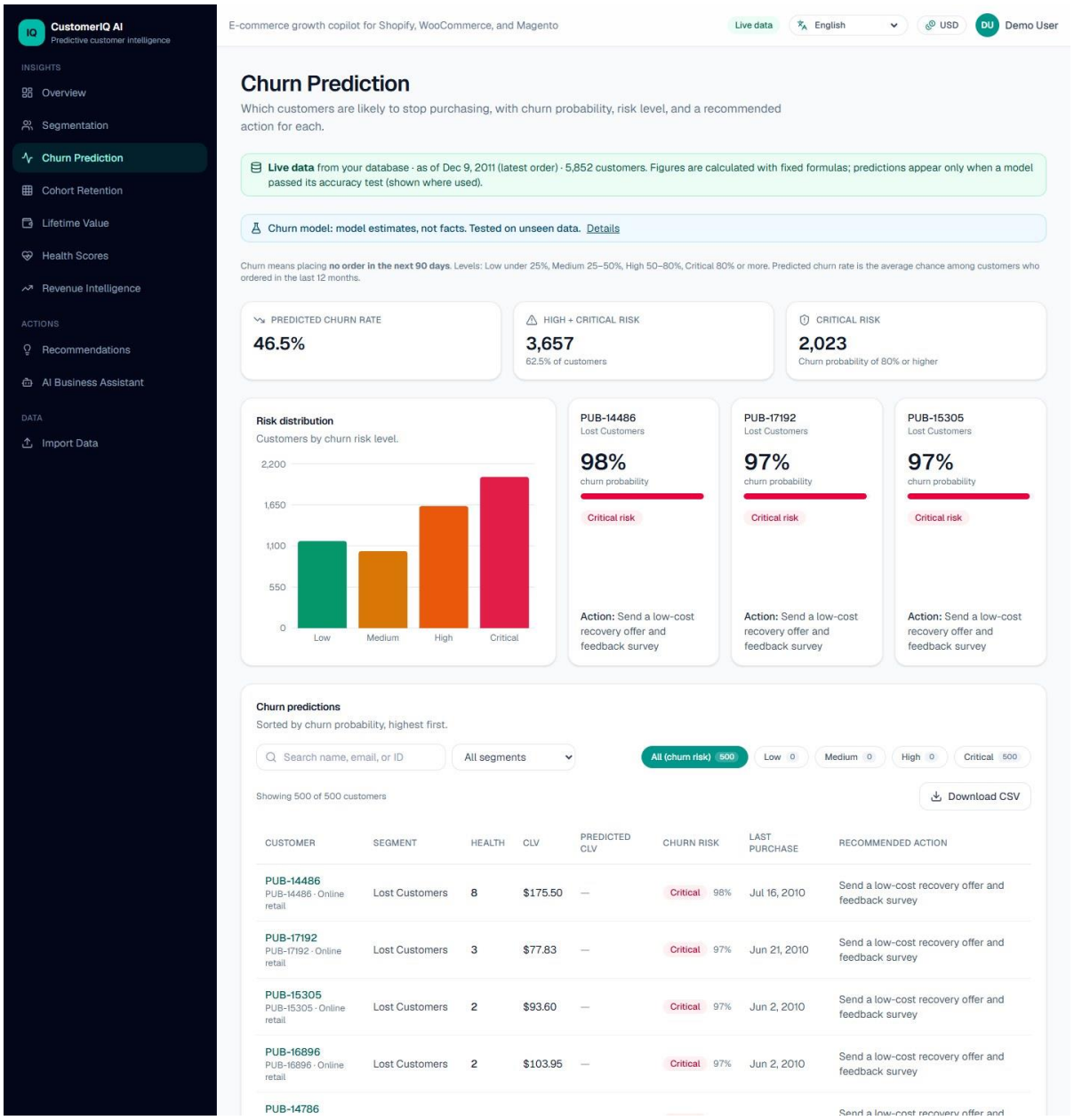


Figure 8.7: Churn prediction page with the model status, risk distribution and customer list

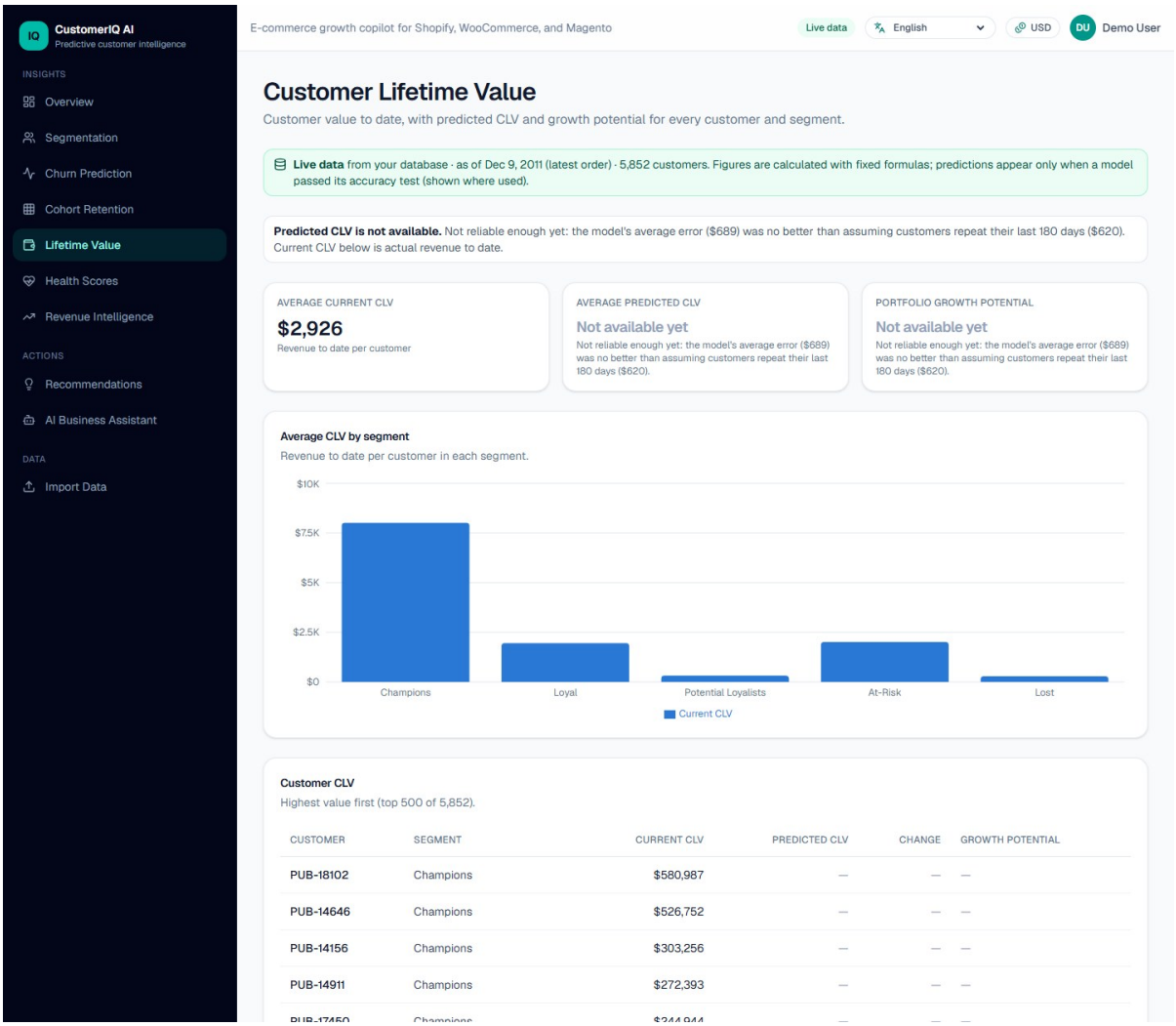


Figure 8.8: Lifetime value page: the prediction is withheld and the reason is shown

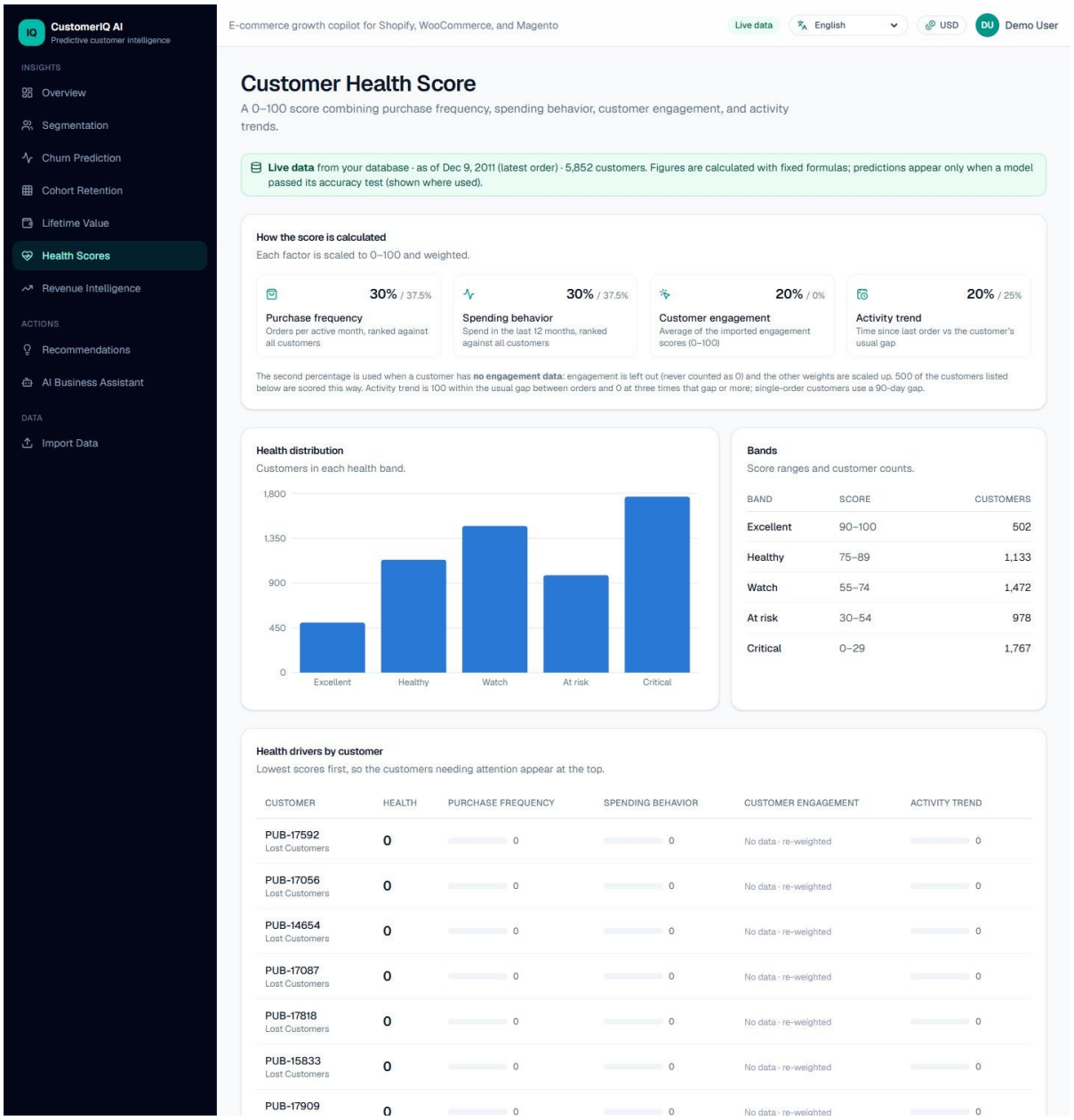


Figure 8.9: Health score page with the formula, distribution and drivers

Revenue, cohorts and recommendations

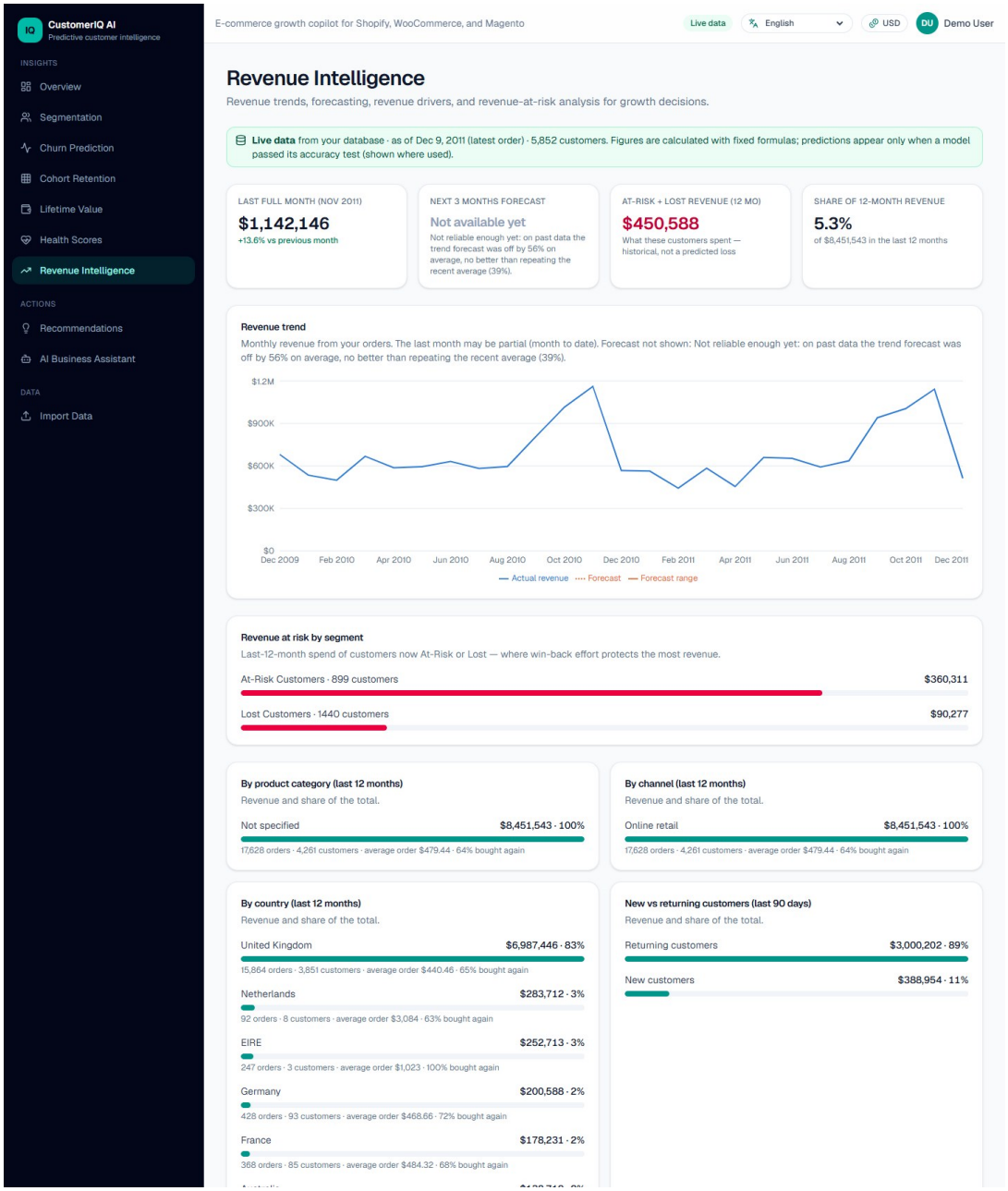


Figure 8.10: Revenue intelligence page: the forecast is withheld and the revenue drivers are shown with order statistics

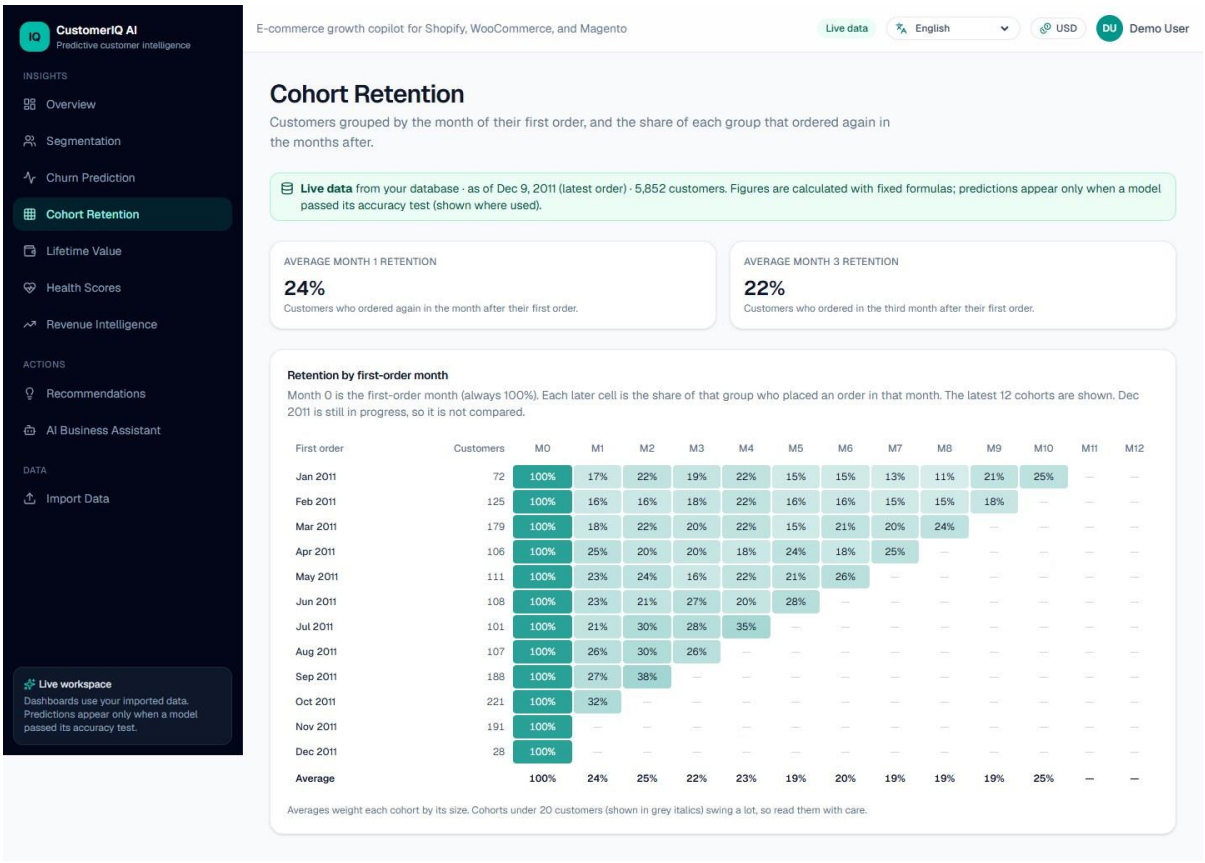


Figure 8.11: Cohort retention table

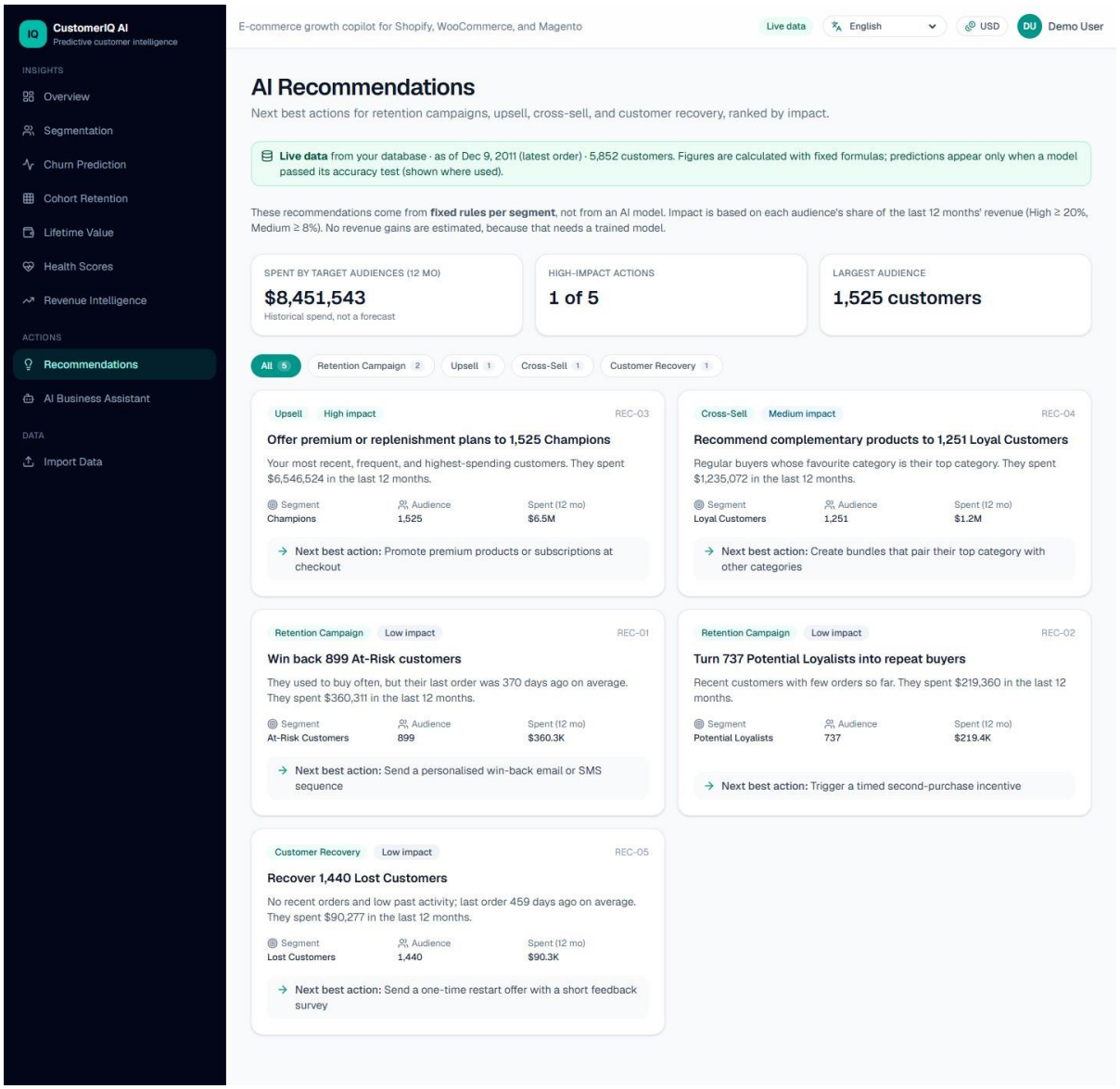


Figure 8.12: Recommendations page

Customer page and import

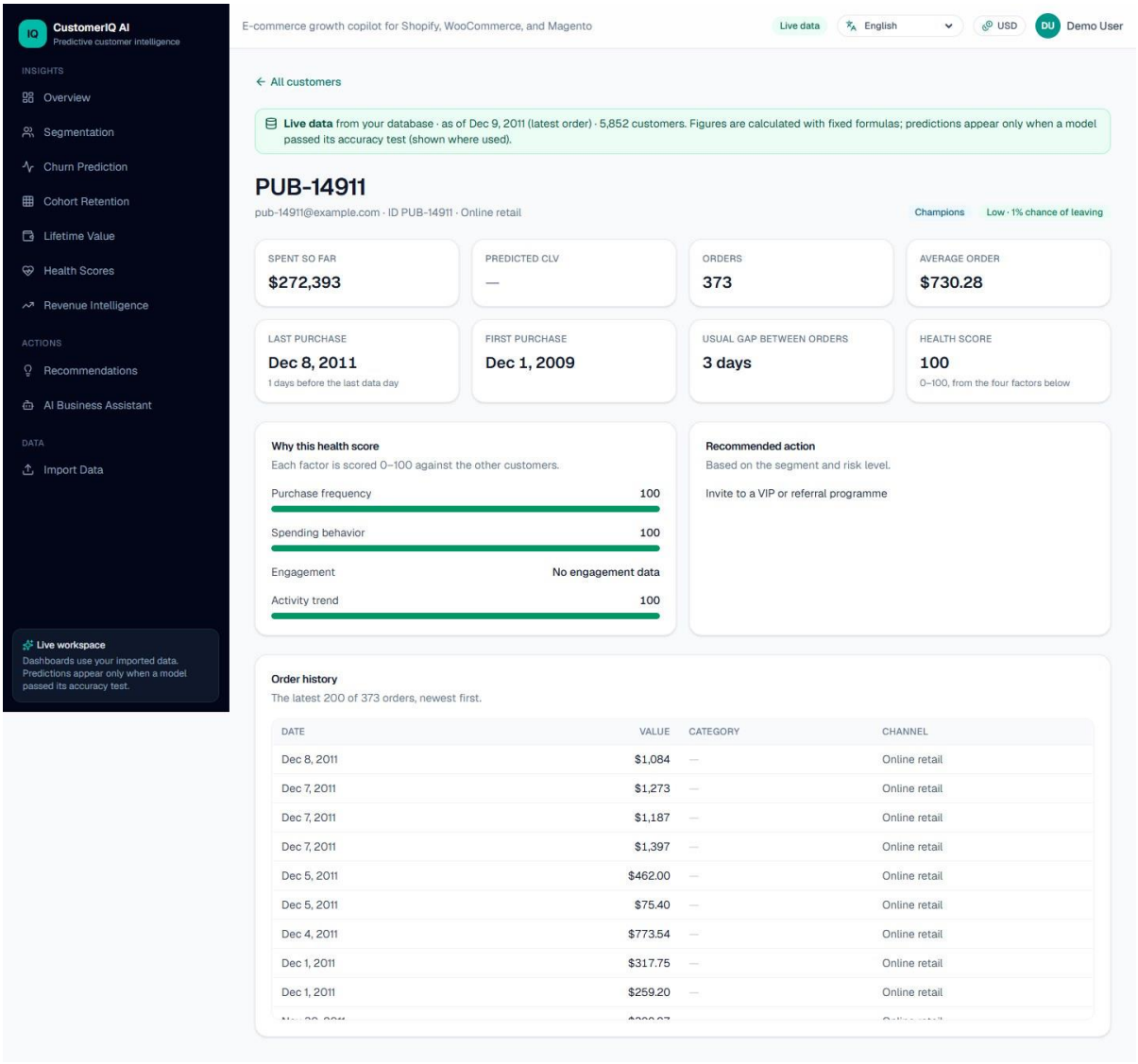


Figure 8.13: Customer page with figures, health factors, recommended action and order history

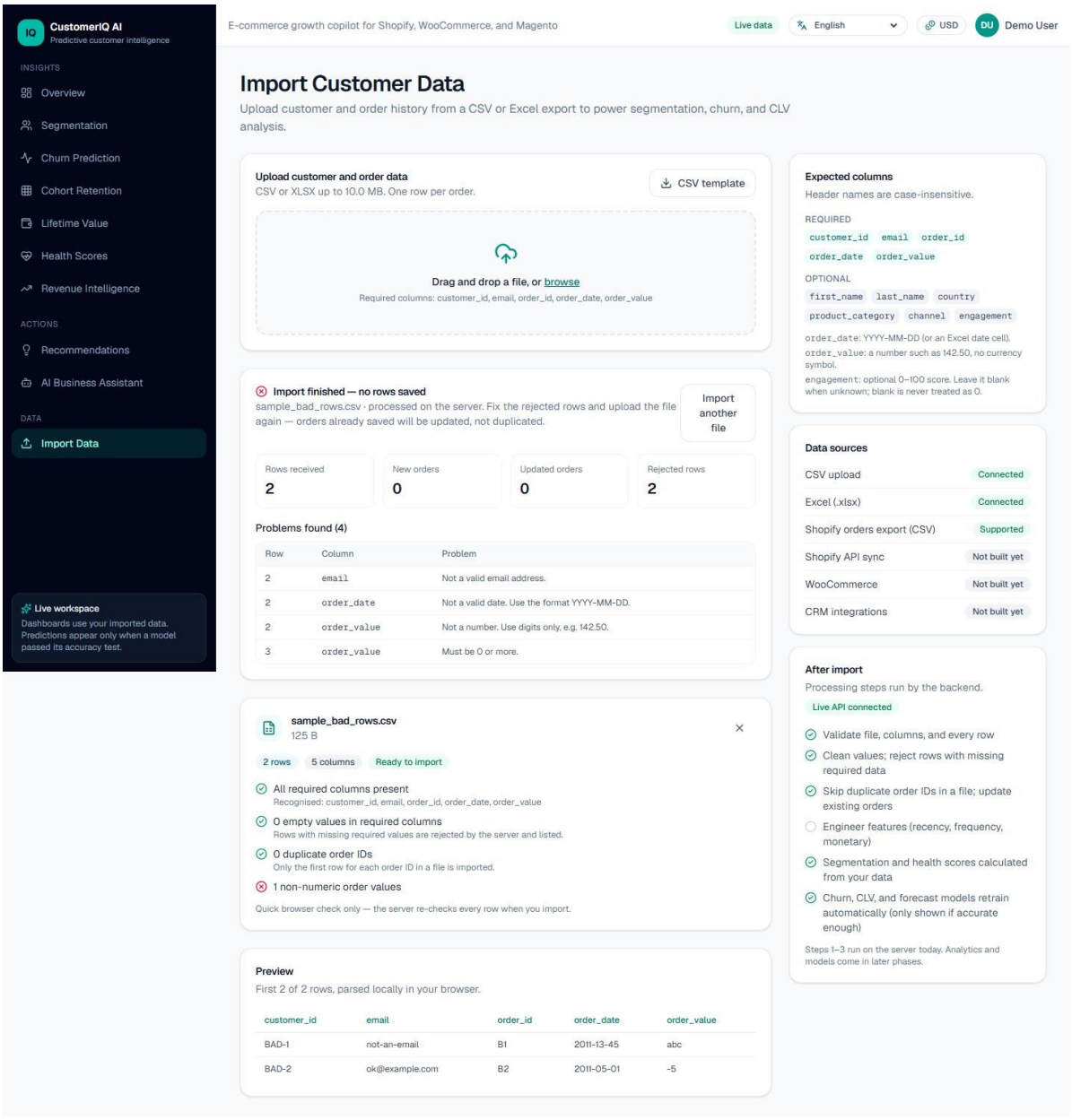


Figure 8.14: Import page after a file with two bad rows: nothing is saved and the problems are listed without repeating the values

Currency, language and phone layout

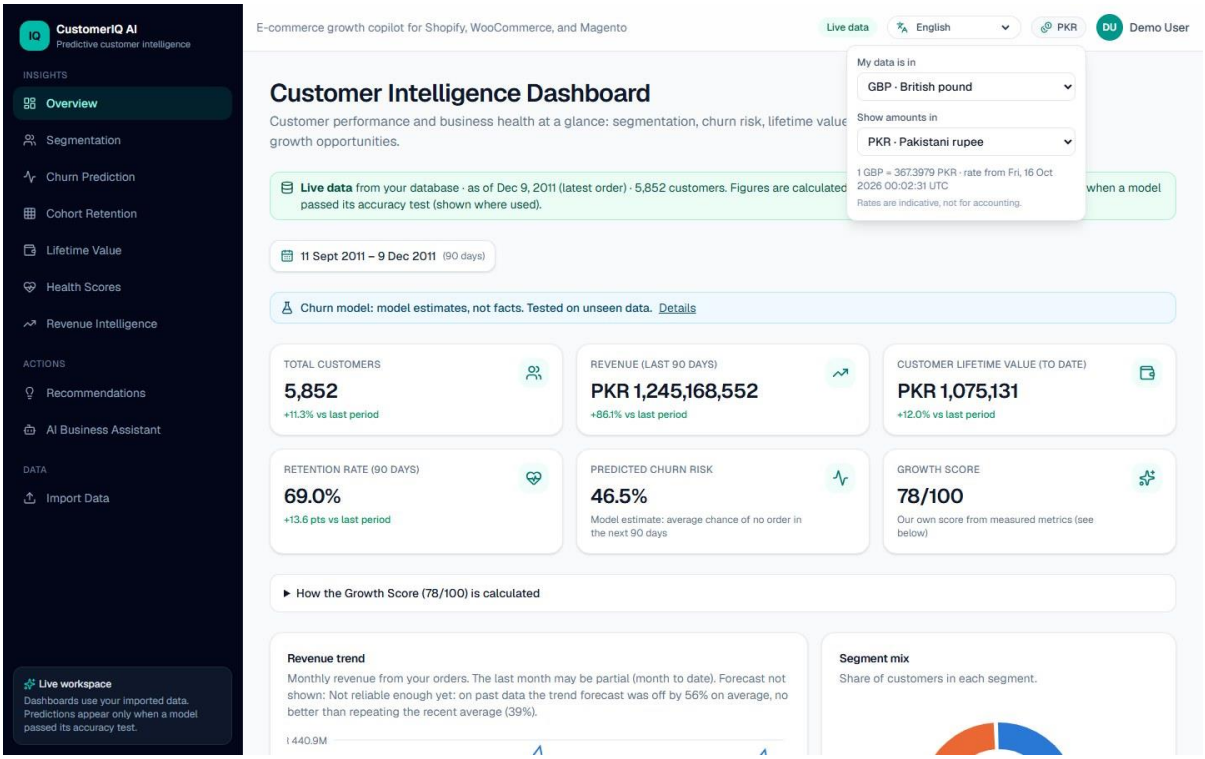


Figure 8.15: Currency menu with data in pounds shown in rupees, with the rate and its date

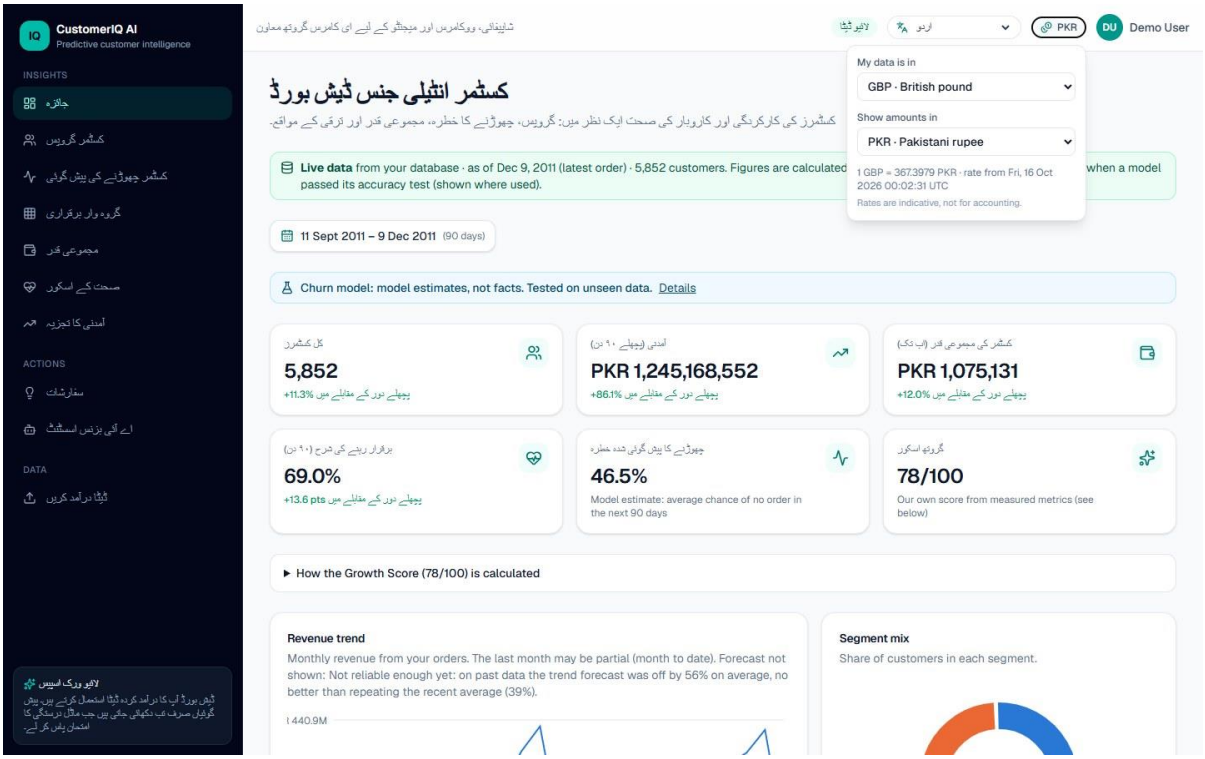


Figure 8.16: Interface in Urdu; long explanatory texts remain in English

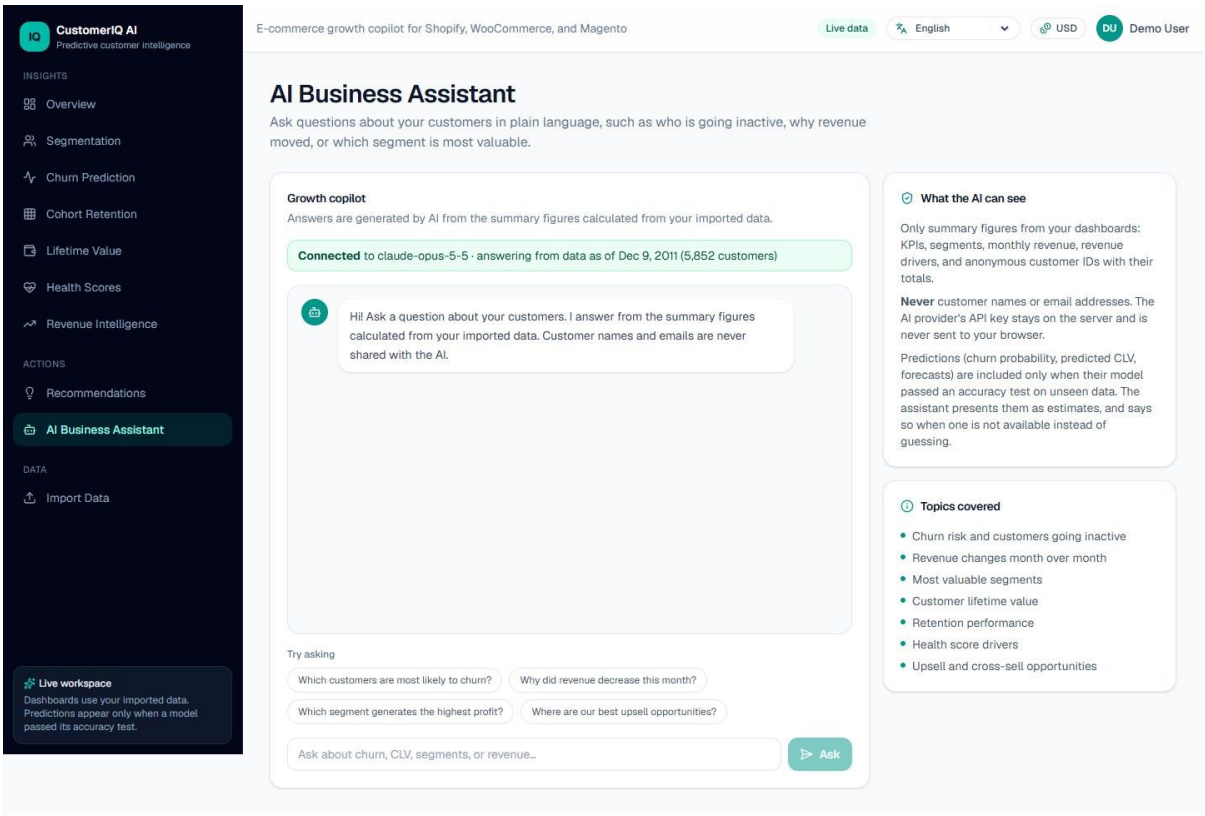


Figure 8.17: AI business assistant page, showing the provider status and what the assistant can see

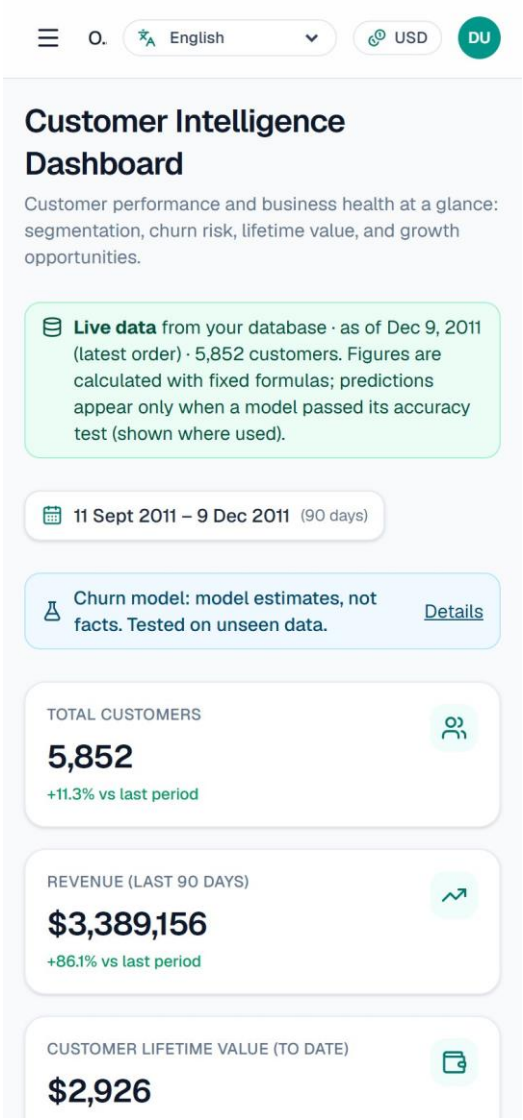


Figure 8.18: Overview on a 390-pixel-wide phone screen

8.5 Security and Access Results

The security controls of Table 5.5 are covered by the tests of Table 7.3 (TC-25 to TC-35, TC-40) and by the browser flows of Table 7.4 (UI-01 to UI-06). In summary, the tests confirmed that data endpoints refuse requests without a session; that wrong credentials give the same message whether or not the account exists; that eight wrong passwords lock an address; that the database holds only hashes of passwords, tokens and codes; that a write from a foreign origin is refused; that the production setting uses secure cookies and hides the API documentation; that a new account cannot see data before its email is confirmed; and that wrong or expired codes are refused after the allowed tries.

These tests show that the controls behave as designed. They are not a security audit: no penetration test was done, and no independent review took place. Two weaknesses are known. The sign-in

lock counter is kept in the memory of the server process, so on a platform that runs several instances at once, each instance counts separately and an attacker gets more tries. Also, because all users share one data set, any confirmed account can see all of the imported data.

8.6 AI Assistant

The assistant was tested with simulated providers (31 tests, Table 7.2), which covered the privacy of the summary, the limits on the conversation, the choice of provider and the mapping of provider errors. On the development computer the assistant page reported a configured provider and the date of the data (Figure in Section 8.4). However, no live question was sent to a provider while this report was prepared, so the quality and correctness of real answers were not evaluated. **[CONFIRM: if you have tested the assistant with real questions, add the questions asked and a short assessment of the answers here]**

8.7 Deployment Results and Problems Met

The website and the API were deployed as two Vercel projects connected to a private GitHub repository, with a Supabase database. The API's health endpoint reported that the API and the database were reachable, and the deployed website served the landing page and the sign-in page. **[CONFIRM: URLs of the deployed website and API, if you want them printed in the report]** Three problems arose and are recorded here because they shaped the design.

- **First build failure.** Vercel stopped the API build because a test configuration file named `pyproject.toml` lacked a project section. The settings were moved to `pytest.ini` and the dependencies were installed from `requirements.txt`.
- **Database migration failure.** The first run of the migration script reached the hosted database but then lost the connection. The script now closes its first connection before migrating and retries, and it reports clearly if the connection keeps failing.
- **Email delivery from the hosted API.** With SMTP settings for a Gmail account, the hosted API could not send the confirmation email. The error was first "Cannot assign requested address" (error 99), and after the connection was forced to use IPv4 only, a time-out. The same account's login was accepted when tested from the development computer, which suggests that the hosted platform does not allow the connection, but the cause was not confirmed with the platform. Two other routes were implemented, Brevo's HTTPS API and a Google Apps Script relay, and both were tested against stand-in servers. At the time of writing neither had been configured or verified on the hosted deployment, so hosted sign-up by email was not working. **[CONFIRM: update this paragraph if the problem has been resolved by the time you submit]**

Two platform limits also affect use. A request to a Vercel Function can carry at most 4.5 MB [15], so the public dataset (3.3 MB) can be uploaded but larger files must be split, although the

application itself accepts files up to 10 MB. The free Hobby plan is restricted to non-commercial, personal use [19], which is why only public test data was placed on it.

8.8 Discussion

Objectives

Table 8.7: Assessment of the objectives

Objective	Assessment	Evidence and remarks
O1 Import with validation	Achieved	36,594 rows imported without rejection; bad rows listed in the browser; 31 import tests.
O2 Descriptive analytics	Achieved	KPIs, segments, health, cohorts, drivers, recommendations on the public dataset; 56 formula and API tests.
O3 Validated predictions	Achieved as designed	Churn model passed (AUC 0.80 against 0.76); the other two were correctly withheld. Accuracy on real store data is unknown.
O4 Assistant	Partly achieved	Implemented and tested with simulated providers; live answers not evaluated.
O5 Access control	Partly achieved	Accounts, sessions, lockout and email confirmation work and are tested; email delivery from the hosted site was unresolved.
O6 Usability	Achieved with limits	Calendar, search, export, currency, twelve languages; only short texts are translated and the layout is not right-to-left.
O7 Free hosting	Achieved with limits	Deployed on free tiers; upload size and email delivery are limited.

Limitations

- All signed-in users share one data set, so the system is unsuitable for confidential data from several stores.
- The customer lists show only the first 500 customers, although the figures above them cover all customers.
- The analytics are recomputed at each request. This is simple and consistent but will not scale to very large stores without caching.
- The sign-in lock counter is per server process.
- The models were evaluated on one public dataset. The lifetime value and forecast models failed their gates on it, so their real performance elsewhere is unknown.
- Interface translations were written for this project and have not been reviewed by native speakers. Urdu and Arabic use the left-to-right layout.
- Exchange rates come from a free public service, are indicative only, and are not historical.
- There is no integration with store platforms or customer relationship management systems.
- The website has no automated unit tests, and no accessibility audit, load test or user study was done.

What was learned

Three lessons stand out. First, making the unavailable state a first-class result, with a reason, changed the design of every page: each component had to handle “not available” as a normal outcome, not an error. Second, the real dataset exposed problems that hand-made test data had not, for example a model that crashed when no customer had an engagement value, which led to dropping empty features before training and to a test for that case. Third, free hosting imposes constraints (upload size, no outgoing mail connections) that are easy to miss when everything has been developed on one computer, and deployment should therefore be tried early.

Chapter 9

CONCLUSION AND FUTURE WORK

9.1 Summary

The project set out to help the owner of a small online store understand their customers from nothing more than an export of their orders, and to be honest about how far the numbers can be trusted. The resulting application, CustomerIQ AI, imports CSV, Excel and Shopify files with row-level validation; groups customers into five RFM segments; calculates health scores, key figures for any chosen period, cohort retention, revenue drivers and recommendations; and shows churn probability, predicted lifetime value and a revenue forecast only when a model has passed a test on a later, unseen period and beaten a simple baseline. Access is protected by accounts, secure sessions and email confirmation, and the application runs on a single computer or on free hosting.

9.2 How the Problem Was Addressed

The problem statement of Chapter 3 named five weaknesses of current practice: manual analysis, dependence on a platform, minimum data conditions, predictions without evidence, and weak models that fail silently. The first three are answered by import-based analytics that need no integration and by models that report their own data shortfall. The last two are answered by the quality gates, which were shown to work on real data: the churn model reached an area under the ROC curve of 0.80 against 0.76 for the baseline, while the spending model and the revenue forecast were withheld with the measured reasons. The evidence consists of 245 passing back-end tests, a browser walk-through of the main flows, and the results on the Online Retail II dataset of 36,594 orders from 5,852 customers.

9.3 Current Limitations

The system should be used with its limits in mind. All signed-in users see one shared data set. Predictions were evaluated on one public dataset only. The interface is translated only for short texts, and the layout is not right-to-left. The AI assistant has not been evaluated with live questions. Email confirmation works in tests and on a local computer but, at the time of writing, delivery from the hosted deployment was unresolved because the hosting platform could not reach the mail server. The website has no automated unit tests, and no accessibility audit, load test or user study was carried out.

9.4 Recommendations and Future Work

The following improvements are listed in the order in which they would matter most for real use.

1. **Separate data for each account.** Introduce workspaces so that every import, customer and order belongs to an account, and enforce it in the database with row-level policies. This is needed before the application can hold the data of more than one business.
2. **Reliable email delivery on the hosted site.** Configure one of the implemented HTTPS routes (Brevo or the Apps Script relay) or a verified sending domain, confirm delivery to a real inbox, and move the sign-in lock counter into the database so that it works across server instances.
3. **Platform integrations.** Add live synchronisation with Shopify and WooCommerce once a test store is available, so that the integration can be verified instead of assumed.
4. **Better forecasting.** Replace the straight-line trend with a model that can follow a yearly pattern, and keep the same gate: it must beat repeating the recent average on past months.
5. **More validation.** Evaluate the models on several datasets and report the results, and check whether the probabilities are well calibrated.
6. **Performance at scale.** Cache the customer snapshot or pre-compute it after each import, and measure the system with larger data and several simultaneous users.
7. **Language and accessibility.** Have the translations reviewed by native speakers, add a right-to-left layout, translate the longer explanations, and run an accessibility audit.
8. **Automated front-end tests and a user study.** Add component and end-to-end tests that run on every change, and observe store owners using the system.

REFERENCES

[1] P. S. Fader, B. G. S. Hardie and K. L. Lee, "'Counting your customers' the easy way: An alternative to the Pareto/NBD model," *Marketing Science*, vol. 24, no. 2, pp. 275–284, 2005, doi: 10.1287/mksc.1040.0098.

[2] D. Chen, S. L. Sain and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *Journal of Database Marketing & Customer Strategy Management*, vol. 19, no. 3, pp. 197–208, 2012, doi: 10.1057/dbm.2012.17.

[3] S. Gupta, D. Hanssens, B. Hardie, W. Kahn, V. Kumar, N. Lin, N. Ravishanker and S. Sriram, "Modeling customer lifetime value," *Journal of Service Research*, vol. 9, no. 2, pp. 139–155, 2006, doi: 10.1177/1094670506293810.

[4] W. Verbeke, K. Dejaeger, D. Martens, J. Hur and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012, doi: 10.1016/j.ejor.2011.09.031.

[5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[8] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950, doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[9] D. Chen, "Online Retail II," UCI Machine Learning Repository, 2019. [Dataset]. Licence: CC BY 4.0. doi: 10.24432/C5CG6D. [Online]. Available: https://archive.ics.uci.edu/dataset/502

[10] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, Internet Engineering Task Force, Aug. 2016. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7914

[11] OWASP Cheat Sheet Series Team, "Password Storage Cheat Sheet," OWASP. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 16, 2026).

[12] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com/ (accessed Oct. 16, 2026).

[13] Next.js documentation, Vercel. [Online]. Available: https://nextjs.org/docs (accessed Oct. 16, 2026).

[14] PostgreSQL Global Development Group, “Row Security Policies,” in *PostgreSQL 18 Documentation*, sec. 5.9. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html (accessed Oct. 16, 2026).

[15] Vercel, “Vercel Functions limits.” [Online]. Available: https://vercel.com/docs/functions/limitations (accessed Oct. 16, 2026).

[16] Klaviyo, “Understanding Klaviyo’s predictive analytics,” Klaviyo Help Center. [Online]. Available: https://help.klaviyo.com/hc/en-us/articles/360020919731-Understanding-Klaviyo-s-predictive-analytics (accessed Oct. 16, 2026).

[17] Shopify, “Creating customer segments,” Shopify Help Center. [Online]. Available: https://help.shopify.com/manual/customers/customer-segmentation/customer-segments (accessed Oct. 16, 2026).

[18] scikit-learn developers, “Ensembles: Gradient boosting, random forests, bagging, voting, stacking,” scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/modules/ensemble.html (accessed Oct. 16, 2026).

[19] Vercel, “Vercel Hobby plan.” [Online]. Available: https://vercel.com/docs/plans/hobby (accessed Oct. 16, 2026).